A Parallel-Scalable Evaluation Algorithm for Regular Path Queries with Counting Operators

Jiangyue Liu, Ting Deng, Jiali Jin, Ping Lu, Haoyi Zhou, Richong Zhang
Beihang University
{liujiangyue, dengting, luping, haoyi, zhangrichong}@buaa.edu.cn, jialijin3@gmail.com

***Abstract*—This paper studies the evaluation of regular path queries with counting operators over graphs. Existing approaches typically rely on linear expansion, which incurs redundant computation, especially for large-scale graphs. We develop Algorithm RPQCEval to address these issues. More specifically, we adopt the hierarchical representation of RPQCs and exploit matrix product to enforce the semantics of counting operators. We develop two optimizations, namely the recursive doubling strategy and common group extraction, to accelerate the computation. To process large-scale graphs, we develop a distributed algorithm to compute the query results and show that it is parallel scalable *w.r.t.* the sequential algorithm RPQCEval, *i.e.,* it reduces the runtime of RPQCEval when more processors are used. Experiments on real and benchmark-derived edge-labeled graphs demonstrate the effectiveness and scalability of the proposed method.**

I. Introduction

Graph data are widely used to represent entities and their relationships, such as RDF knowledge graphs [1], social networks [2] and biological networks [3]. Regular path queries (RPQs) are the basic query mechanism for expressing path patterns over edge-labeled graphs [4], [5]. Intuitively, a regular path query is a regular expression defined on graphs. Given a graph G, a regular path query is to find all vertex pairs (u, v) for which there exists a path between u and v such that the edge labels on the path form a string in the language specified by the regular path query (*i.e.,* the regular expression).

Existing algorithms, *e.g.,* [6]–[11] for evaluating RPQs mainly focus on standard RPQs, *i.e.,* using only union, concatenation and Kleene star. However, many applications require not only reachability but also path patterns with specified repetition bounds. For example, $\mathsf{parent}\{2,4\}/\mathsf{birthPlace}$ finds person–place pairs connected by two to four consecutive parent edges followed by one birthPlace edge. We use this query as a running example throughout the paper. Such queries can be expressed using counting regular path queries, where $B\{k\}$ means that a path body B is repeated exactly k times, which is also supported by the ISO standards [5].

Existing approaches for counting RPQs can be roughly divided into two categories. The first category is query rewriting, which rewrites counting expressions into equivalent standard regular expressions before evaluation. For example, $R\{m\}$ can be rewritten as $R \cdot R \cdots R$. After rewriting, the query can be evaluated by existing regular-expression or property-path engines. This strategy is simple and compatible with standard RPQ evaluation techniques, but the rewritten RPQs may grow exponentially with repetition bound, which increases the size of the query plan or the equivalent automaton. For example, rewriting a RPQ with counting operator $((ab)\{1000\})\{1000\}$ results in a RPQ $ab{\cdot}\ldots{\cdot}ab$ with ab repeated for 1000,000 times.

The second category is extended-state automata [12]–[14], like counting-set and bit-vector automata, which can compactly represent counting operators. Instead of expanding RPQCs syntactically, these automata extend state spaces of a finite automaton with explicit counting information. A state is represented by not only a graph vertex and an automaton state, but also a counter. A transition in the automaton updates both the state and the counter, and a transition is allowed only when the counter satisfies the counting constraints. This method avoids full expansion, but may match the same subquery on the same path for multiple times, leading to redundant computation. Consider the above query $((ab)\{1000\})\{1000\}$; when the graph contains a cycle $v_1 a v_2 b v_1$, these methods would visit the cycle for 1000,000 times, which is redundant.

Is it possible to exploit automata to match RPQs with counting operators, without expanding the counting operators and repeatedly matching the same paths?

Contributions. We answer this question affirmatively by developing an algorithm that combines automaton methods with matrix product to handle counting operators.

More specifically, we make the following contributions.

(1) Hierarchical and incremental evaluation. Given a RPQC Q, we decompose it into a hierarchical collection of RPQs. We replace each counting operators $R\{m, n\}$ by a symbol $C_{R\{m,n\}}$, construct extended automata $\mathcal{A}_e$ using such symbols, and evaluate RPQC by traveling $\mathcal{A}_e$ from every vertex in G. When reaching symbol $C_{R\{m,n\}}$, we (1) first recursively compute $R(G)$ using an automaton defined by R, starting from only vertices reached during traveling $\mathcal{A}_e$, and (2) then compute $R\{m, n\}(G)$ using matrix product, and continue to travel $\mathcal{A}_e$ from only reached vertices in $R\{m, n\}(G)$.

(2) Sequential algorithm. We propose a sequential algorithm to compute $Q(G)$ for RPQC Q. To compute counting operators $R\{m, n\}(G)$, we extract common groups from $R(G)$ to share computation during matrix product. We show that maximizing revenue of common groups is NP-complete, and develop a heuristic algorithm for it, which iteratively generates groups by considering the vertex with the maximum incoming edges and build inverted indices to extract vertex pairs. The algorithm has lower complexity than existing grouping strategies like

bicliques [15], and strikes a balance between efficiency and effectiveness of common groups (see Section V).

(3) Distributed algorithm. To process large-scale graphs, we develop a distributed algorithm to compute $Q(G)$. We partition G via edge-cut to evenly distribute automaton traveling for each source vertex in G. For matrix product, we first redistributed the workload by hashing vertex IDs before the computation, and exploit work stealing to move computation from overload workers to underloaded workers during the computation. We show that the distributed algorithm is parallel-scalable *w.r.t.* the sequential algorithm, *i.e.,* it reduces the runtime of Evaluate when more processors are used.

(4) Empirical evaluation. Using real-life and synthetic graphs, we evaluated the performance of RPQCEval. We find the following: (1) RPQCEval handles large graphs efficiently. It takes 203.648s to process graph SF3 with 9.282M vertices and 52.696M edges. (2) It outperforms all baselines on every dataset, with $6.75\times$–$35.78\times$ speedup over the strongest baseline on each dataset. (3) The optimizations are effective; the recursive doubling strategy and common group extraction achieves speedups of $2.57\times$–$4.24\times$ and $1.36\times$–$2.12\times$, respectively. (3) RPQCDEval is parallel scalable with respective to RPQCEval. RPQCDEval achieves speedups of $7.76\times$–$11.50\times$ when the number of workers varies from 2 to 32.

Related Work. We categorize related work as follows.

Query languages on graphs. Research on query languages on graphs has mainly followed three directions. (1) RPQs and its extensions: (i) the semantics and complexity of RPQs are extendedly studied [6]–[11]; and (ii) other extensions include: (a) regular queries [16], which extends UC2RPQs and UCN2RPQs with transitive closure; (b) positive 2-way regular path queries in Description Logics [17]; and (c) conjunctive regular path queries [18]. (2) Varies practical problems for RPQs, including output-sensitiveness of RPQs [19], the minimization problem for Conjunctive Regular Path Queries (CRPQs) and unions of CRPQs (UCRPQs) [20], the resilience for Regular Path Queries [21], and approximate answers of RPQs [22]. (3) Evaluation problems over streaming graphs [23]–[25] or temporal property graphs [8], [26].

Algorithms for RPQs. Research on algorithms to evaluate RPQs on graphs has mainly followed three directions: (1) auxiliary data structure to accelerate automaton travel, *e.g.,* RPQ-based graph reduction [27], node clustered index [28], recursive index tree [29], views [30], [31] and bit-parallel simulation and the ring index [32]; (2) matrix product to evaluate queries, *e.g.,* the matrix algebra for evaluating RPQs [33], the context-free path querying [34], single-source regular path querying [35] and SPARQL queries [36]; (3) GPUs for conjunctive regular path queries [37]; (4) studying subclasses, *e.g.,* [38] develops efficient algorithms for transitive restricted expression using the simple path semantics; and (5) benchmark RPQBench to evaluate RPQ algorithms [39], [40].

Unlike prior work, we focus on regular path queries with counting operators. To this end, we interleave the automaton traveling and matrix product to handle counting operators. We studied the maximum revenue problem, which aims to reduce redundant computaition in matrix product, and develop an algorithm for the problem, which complexity is lower than similar solution for Boolean matrix production [15]. To avoid overloaded workers in distributed setting, we redistributed workload before matrix production, and move computation from overload workers to underloaded workers during computation.

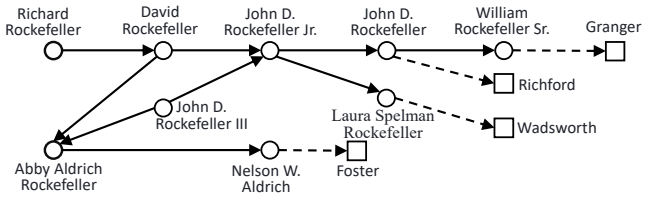


Fig. 1: A graph extracted from DBpedia. Circles and squares are persons and locations, respectively. Solid and dashed arrows denote parent and birthPlace relationships, respectively.

II. Preliminaries

In this section, we introduce edge-labeled graphs, regular path queries and regular path queries with counting.

Edge-labeled graphs. Let Γ be the set of edge labels. We consider an *edge-labeled directed graph* $G = (V, E, \Gamma)$, where V is a finite set of vertices, $E \subseteq V \times \Gamma \times V$ is a finite set of edges, and each edge $e = \langle u, r, v\rangle$ denotes an edge from vertex u to v and is labeled with $r \in \Gamma$.

Given an edge-labeled directed graph $G = (V, E, \Gamma)$ and two vertices u and v in G, a *path* ρ from u to v is a sequence $v_0 e_1 v_1 e_2 v_2 \cdots v_k e_k v_{k+1}$, where (1) $v_0{=}u$ and $v_{k+1}{=}v$; and (2) for each $i \in [1, k]$, $e_i{=}\langle v_{i-1}, r_i, v_i\rangle$ is an edge in E labeled r_i. Denote by $L(\rho)$ the *label sequence* $r_1 r_2 \cdots r_k$ on path ρ.

Example 1: Figure 1 shows an edge-labeled graph extracted from DBpedia, consisting of 13 vertices and 13 edges. The vertices represent persons or places, while solid and dashed arrows denote parent and birthPlace relationship, respectively. Consider path ρ from DavidRockefeller to Richford that passes through vertices JohnD.Rockefeller and JohnD.Rockefeller, following two parent edges and one birthplace edge. Thus, its label sequence is $L(\rho) = \mathsf{parent} \cdot \mathsf{parent} \cdot \mathsf{birthPlace}$. □

Regular Path Queries with counting. A regular path query with counting (RPQC) is recursively defined as follows:

$$R := \varepsilon \mid a \mid (R_1|R_2) \mid R_1 \cdot R_2 \mid R\{m, n\}, \qquad (1)$$

where (1) empty string ε is an RPQC; (2) a ($a \in \Gamma$) is an RPQC; (3) if R_1 and R_2 are RPQC, then $R_1|R_2$ (union) and $R_1 \cdot R_2$ (concatenation) are RPQCs; and (4) if R is an RPQC, m is a natural number (*i.e.,* $m \in \mathbb{N}$) and $n \in \mathbb{N} \cup \{\infty\}$ with $m \leq n$, then $R\{m, n\}$ is also an RPQC.

Here, $\{m, n\}$ is called counting operator, and $R\{m, n\}$ is $R \ldots R(\varepsilon|R) \ldots (\varepsilon|R)$, where $R \ldots R$ is m repetitions of R followed by $n - m$ repetitions of $(\varepsilon|R)$. In particular, $R\{m\}$

is an abbreviation of $R\{m,m\}$ representing *exact counting*, *i.e.,* exactly m repetitions of R; $R\{m,n\}$ with $m<n<\infty$ represents *bounded-range counting*; and $R\{m,\infty\}$ denotes *unbounded counting*, allowing at least m repetitions of R [13]. Moreover, the Kleene star operator is a special case of counting, as R^* is equivalent to $R\{0,\infty\}$.

Semantics. Given a RPQC Q, its language $\mathcal{L}(Q)$, *i.e.,* the set of strings denoted by Q, is defined inductively as follows: (a) if Q is empty string ε or a ($a\in\Gamma$), then $\mathcal{L}(Q)=\{\varepsilon\}$ or $\mathcal{L}(Q)=\{a\}$; (b) if Q is in the form of $R_1|R_2$, then $\mathcal{L}(Q)=\mathcal{L}(R_1)\cup\mathcal{L}(R_2)$; (c) if Q is in the form of $R_1\cdot R_2$, then $\mathcal{L}(Q)=\mathcal{L}(R_1)\cdot\mathcal{L}(R_2)$, *i.e.,* $\mathcal{L}(Q)=\{ab \mid a\in\mathcal{L}(R_1)\wedge b\in\mathcal{L}(R_2)\}$; and (d) if Q is in the form of $R\{m,n\}$, then $\mathcal{L}(Q)=\bigcup_{i=m}^{n}\mathcal{L}(R)^i$, where $\mathcal{L}(R)^i$ is the i-fold concatenation of $\mathcal{L}(R)$, *i.e.,* $\mathcal{L}(R)^i=\{l_1\dots l_i \mid l_1,\dots,l_i\in\mathcal{L}(R)\}$, with $\mathcal{L}(R)^0=\{\varepsilon\}$.

Given a graph $G=(V,E,\Gamma)$ and an RPQC Q, the query result $Q(G)$ is the set of all vertex pairs $(u,v)\in V\times V$ such that there exists a path ρ from u to v satisfying $L(\rho)\in\mathcal{L}(Q)$. We refer to u and v as *source vertex* and *target vertex*, respectively. We adopt the arbitrary-path semantics of RPQCs [41], under which a vertex may appear multiple times on a path.

Example 2: Continuing with Example 1, consider the RPQC $Q=\mathsf{parent}\{2\}\cdot\mathsf{birthPlace}$, which retrieves birthplaces of a person's grandparents by following exactly two parent edges and one birthPlace edge. For example, the vertex pair $(\mathsf{DavidRockefeller},\mathsf{Richford})$ is an result to Q. On the graph in Figure 1, the query Q returns seven distinct vertex pairs.

Consider another RPQC $Q'=\mathsf{parent}\{2,4\}\cdot\mathsf{birthPlace}$, which extends Q by allowing two to four parent edges, enabling the analysis of family's geographic origins across generations. For example, from DavidRockefeller, Q' retrieves Richford, Wadsworth, Foster and Granger. On the graph in Figure 1, Q' returns 13 distinct vertex pairs. □

Matrix representations. Given an RPQ R and a graph $G=(V,E,\Gamma)$, the query result $R(G)$ can be represented as a Boolean matrix M_R, whose rows and columns are indexed by vertices in V. Specifically, $M_R[u,v]=1$ if $(u,v)\in R(G)$, and 0 otherwise. Given RPQs R and R', query result $R(G)\cdot R'(G)$ can be computed by Boolean matrix multiplication:

$$M_{R\cdot R'}[u,w]=\bigvee_{v\in V}\big(M_R[u,v]\wedge M_{R'}[v,w]\big), \qquad (2)$$

where $M_T[u,v]=1$ indicates that there exists a path from u to v whose label sequence belongs to $\mathcal{L}(T)$.

Problem definition. We study the following RPQC evaluation problem: *Given a graph $G=(V,E,\Gamma)$ and an RPQC Q, it outputs the query result $Q(G)$.*

Complexity. Even for RPQs, computing $Q(G)$ cannot be done in $O((|V|\cdot\max(|V|,E))^{1-\varepsilon}\cdot f(|q|))$ time for any function f and constant $\varepsilon\geq 1$; otherwise, the Boolean Matrix Multiplication (BMM) assumption fails [10]. The BMM assumption states that for any $\varepsilon>0$, no algorithm can multiple two $n\times n$ Boolean matrices in $O(n^{3-\varepsilon})$ time. For real-life graphs, the number of edges is typically larger than the number of vertices, *i.e.,* $|V|\leq|E|$. Thus, $Q(G)$ cannot be computed in $O((|V|\cdot|E|)^{1-\varepsilon}\cdot f(|q|))$ time under the BMM assumption.

In the following, we develop an algorithm that compute $Q(G)$ in $O(|Q||V|^3)$ time. which is "almost" optimal given the above lower bound. Indeed, $|V||E|\leq|V|^3$ and when the graph is dense, *e.g.,* the graph is complete, $|V||E|$ is close to $|V|^3$. Here, $|Q|$ is the size of the query Q, defined as the total number of symbols and the operators in Q plus the total bit length of the integer bounds in the counting operators in Q [42].

III. Sequential Algorithm for RPQCs

In this section, we develop a sequential algorithm for evaluating RPQC, whose framework is given in Figure 2.

A. Challenges and solutions

Challenges. We need to address the following challenges.

(1) Existing matrix-based and automaton-based methods have limitations in handling counting operators. Matrix-based methods cannot effectively exploit the adjacent relationships between edges in the graph to accelerate the evaluation, while automaton-based methods may repeatedly traverse the same paths to handle the counting operators. How can we combine both methods to efficiently evaluate RPQCs?

(2) The counting operators $R\{m,n\}$ ($m\leq n$) computes the union of $R\{m\}(G),R\{m+1\}(G),\dots,R\{n\}(G)$. However, computing these sets separately is costly and may contain redundant computations, especially when the gap beteen the two bounds, *i.e.,* $n-m$, is large. How can we reduce such redundancy and accelerate the evaluation?

(3) Given the results of $R(G)$, evaluating $R\{m,n\}(G)$ involves Boolean matrix multiplication. Specifically, we need to efficiently compute $R_1(G)\cdot R_2(G)$ for two RPQs R_1 and R_2, where $R_1(G)$ and $R_2(G)$ are represented as Boolean matrices. Since these query results may contain vertex pairs that are not edges in G, the adjacency relationships in the original graph G cannot be directly exploited to accelerate the computation. Moreover, directly computing $R_1(G)\cdot R_2(G)$ using matrix multiplication is costly when G is large. How can we accelerate this matrix multiplication ?

To address the challenges, we use the following strategies.

Hierarchical representation of RPQCs. We develop a hierarchical representation of RPQCs and a recursive evaluaion strategy based on this representation.

Hierarchical representation. Given an RPQC Q, we recursively decompose it into a hierarchical collection of counting-free RPQs while preserving the nesting structures of its counting operators. Specifically, we construct Q_{cf} by replacing each *outermost* counting operator $R\{m,n\}$ in Q with a new symbol $C_{R\{m,n\}}$ not appearing in Γ. We then recursively apply the same procedure to the body R of each counting expression, obtaining its counting-free form R_{cf}.

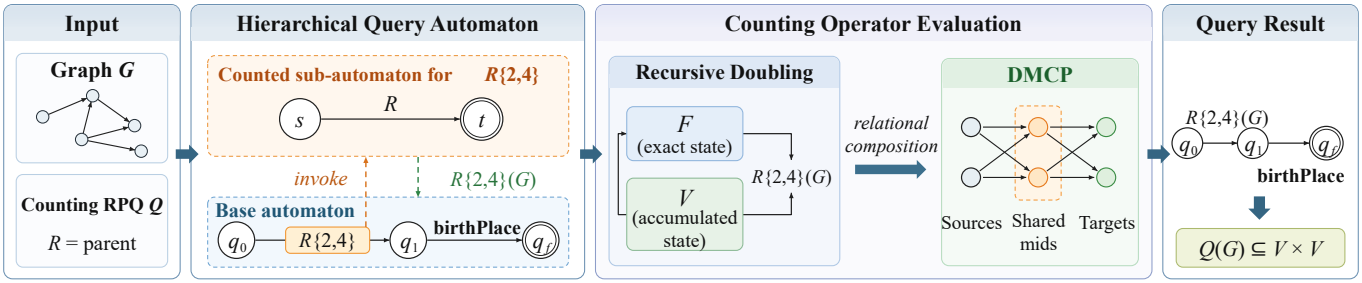


Fig. 2: The framework of RPQCEval. Given a graph G and a RPQC Q, RPQCEval computes $Q(G)$ in the following steps: (1) construct the hierarchical representation of the RPQC Q; (2) compute $Q(G)$ recursively by traveling the automata of these hierarchical representation; (a) when it reaches a symbol in Γ, it continues to travel the automata; (b) if it reaches a symbol representing counting operators $R\{m,n\}$, it first computes the result $R(G)$, from which it computes $R\{m,n\}$ using common group extraction and the recursive doubling strategy; and (c) when it reaches an accepting state, it extends final results $Q(G)$.

This process produces a hierarchy of counting-free RPQs, each of which can be represented by a automaton.

Example 3: Consider query $Q' = \mathsf{parent}\{2,4\} \cdot \mathsf{birthPlace}$ from Example 2 on the graph in Figure 1. We replace counting expression $\mathsf{parent}\{2,4\}$ with $C_{\mathsf{parent}\{2,4\}}$, obtaining counting-free RPQ Q'_{cf}=$C_{\mathsf{parent}\{2,4\}} \cdot \mathsf{birthPlace}$. Since parent contains no counting operator, no further decomposition is needed. □

Recursive evaluation of RPQCs. Given graph G and RPQC Q, we compute $Q(G)$ by recursively traversing the automata of counting-free RPQs obtained by hierarchical decomposing Q.

(a) We construct the automaton of Q_{cf}. Transitions labeled by symbols in Σ are processed as in standard RPQ evaluation, whereas transitions labeled by $C_{R\{m,n\}}$ trigger the evaluation of the corresponding counting expression $R\{m,n\}$.

(b) We traverse the automaton Q_{cf} from its initial state of the automaton for Q_{cf}, until a final state is reached. When a transition labeled by $C_{R\{m,n\}}$ is encountered, we evaluate $R\{m,n\}$ only from the vertices in G that have reached the symbol $C_{R\{m,n\}}$, rather than from every vertex in G.

(c) When evaluating $R\{m,n\}(G)$, we iteratively extend the previously computed results in $R(G)$. Specifically, only vertices that can reach the transion labeled by symbol $C_{R\{m,n\}}$ are used to compute $R(G)$. Let V_I^R denote these vertices, and V_R denote the set of vertices reachable from vertices in V_I^R through paths matching R. we iteratively extend the $R(G)$ by evaluating R from vertices in V_R whose results have not yet been computed. This process terminates when no new vertices can be reached. We then compute $R\{m,n\}(G)$ using matrix multiplication based on the recursive doubling strategy (see below).

Example 4: Continuing with Example 3, for the graph in Figure 1 and the query Q' in Example 2, we first evaluate the counting-free RPQ $Q'_{\mathsf{cf}} = C_{\mathsf{parent}\{2,4\}} \cdot \mathsf{birthPlace}$. since its first symbol is $C_{\mathsf{patent}\{2,4\}}$, we recursively compute $\mathsf{parent}(G)$ using the automaton of RPQ parent, and then compute $\mathsf{parent}\{2,4\}(G)$ through matrix multiplication.

With David Rockefeller as the source vertex, $\mathsf{parent}\{2,4\}(G)$ contains four vertex pairs: (David Rockefeller, John D. Rockefeller), (David Rockefeller, Laura Spelman Rockefeller), (David Rockefeller, Nelson W. Aldrich) and (David Rockefeller, William Rockefeller Sr.). We then continue evaluating birthPlace from these intermediate vertices to obtain the results of Q'. □

Recursive doubling strategy. We design a recursive doubling strategy to efficiently evaluate the counting operators $R\{m,n\}$. Observe that (1) $R\{m,n\}$ can be rewritten as $R\{m\} \cdot R\{0,n-m\} = R\{m\} \cdot (\varepsilon|R\{1\}|\ldots|R\{n-m\})$; (2) the results of $R\{m\}$, $R\{1\},\ldots,R\{n-m\}$ can be computed by composing the results of $R, R\{2\},\ldots,R\{2^k\}$, where $k = \lfloor \log_2 n \rfloor$; and (3) the union $(\varepsilon|R\{1\}|\ldots|R\{n-m\})(G)$ can be computed incrementally by reusing previously computed results; *e.g.*, $(\varepsilon|R|R\{2\}|R\{3\})(G) = (\varepsilon|R\cdot(\varepsilon|R\cdot(\varepsilon|R)))(G)$.

Assume that $2^{k_L} \leq m < 2^{k_L+1}$ and $2^{k_U} \leq n < 2^{k_U+1}$, where $m \leq n$. Let $k_{\max} = \max(k_L, k_U) + 1$. We recursively construct auxiliary RPQs F^i and V^i ($i \in [1, k_{\max}]$) as follows.

(1) for $i = 1$, we set $F^1 = R$ and $V^1 = \varepsilon$; and

(2) for $i \in [2, k_{\max}]$, we compute $F^i = F^{i-1} \cdot F^{i-1}$ and $V^i = V^{i-1}|(V^{i-1} \cdot F^{i-1})$. By induction, we can show that $F^i = R\{2^{i-1}\}$ and $V^i = \varepsilon \mid R \mid \ldots \mid R\{2^{i-1}-1\}$.

Using these auxiliary RPQs F^i and V^i ($i \in [1, k_{\max}]$), we compute $R\{m,n\}$ by combining them according to the binary representations of m and $n-m+1$, respectively, without seperately computing $R\{j\}(G)$ for every $j \in [m,n]$.

Moreover, the recurrence of V_i enables incremental evaluation of $R\{m,n\}$. Since $V^i = V^{i-1} \mid (V^{i-1} \cdot F^{i-1})$, we have $V^{i-1}(G) \subseteq V^i(G)$, and thus $V^i(G)$ grow monotonically as i increases. Then we do not need to evaluate $V^{i-1} \cdot F^{i-1}$ for vertex pairs (w,u) if (a) either (w,u) is already in V^{i-1}, or (b) there is (w,v') and (v',u) in V^{i-1} and $F^{i-1}(G)$, respectively, since no new vertex pairs is generated in these cases.

Example 5: We show how to compute counting operators using the recursive doubling strategy.

Consider the counting expression $\mathsf{patent}\{2,4\}$ in queries

from Example 2. Then $\mathsf{patent}\{2,4\} = \mathsf{patent}\{2\} \cdot (\varepsilon|\mathsf{patent}|\mathsf{patent}\{2\})$. We compute $\mathsf{patent}\{2,4\}(G)$ using the auxiliary RPQs shown in Table I. In particular, F^1, F^2 and F^3 are used to compute $\mathsf{patent}(G)$, $\mathsf{patent}^2(G)$ and $\mathsf{patent}^4(G)$, respectively; and V^1, V^2 and V^3 are used to compute $\varepsilon(G)$, $(\varepsilon|\mathsf{patent})(G)$ and $(\varepsilon|\mathsf{patent}|\mathsf{patent}\{2\}|\mathsf{patent}\{3\})(G)$, respectively; *e.g.,* $V^2 = V^1|(V^1 \cdot F^1) = \varepsilon|\varepsilon \cdot \mathsf{patent} = \varepsilon|\mathsf{patent}$.

TABLE I: Recursive doubling of $\mathsf{parent}\{2,4\}$

i	F^i	V^i
1	patent	$V^1 = \varepsilon$
2	$\mathsf{patent}\{2\}$	$V^2 = V^1\|(V^1 \cdot F^1) = \varepsilon\|\mathsf{patent}$
3	$\mathsf{patent}\{4\}$	$V^3 = V^2\|(V^2 \cdot F^2) = \varepsilon\|\mathsf{patent}\|\mathsf{patent}\{2\}\|\mathsf{patent}\{3\}$

Given Table I, the RPQC $\mathsf{patent}\{2,4\}$ can be rephrased as $F^2 \cdot (F^2 \cdot V^1|V^2) = \mathsf{patent}\{2\} \cdot (\varepsilon|\mathsf{patent}|\mathsf{patent}\{2\}) = \mathsf{patent}\{2\}|\mathsf{patent}\{3\}|\mathsf{patent}\{4\}$.

Consider another query $\mathsf{patent}\{3,8\}$=$\mathsf{patent}\{3\}\cdot(\varepsilon|\mathsf{patent}|\ldots|\mathsf{patent}\{5\})$. Given Table I, since the binary encodings of 3 and 5 are 11 and 101, respectively, the RPQC $\mathsf{patent}\{3,8\}$ can be rephrased as $F^1 \cdot F^2 \cdot (F^3 \cdot V^2|V^3) = \mathsf{patent}\{1\} \cdot \mathsf{patent}\{2\} \cdot (\mathsf{patent}\{4\} \cdot (\varepsilon|\mathsf{patent})|\varepsilon|\mathsf{patent}|\mathsf{patent}\{2\}|\mathsf{patent}\{3\}) = \mathsf{patent}\{3\} \cdot (\varepsilon|\mathsf{patent}|\ldots|\mathsf{patent}\{5\})$. □

Remark. For an RPQC of the form $R\{m,\infty\}$, we rewrite it as $R\{m\}\cdot R\{0,\infty\}$=$R\{m\}\cdot R^*$. Then we compute $R\{m\}(G)$ using the recursive doubling strategy, and compute $R\{0,\infty\}(G)$ (*i.e.,* $R^*(G)$) by computing transitive closure of $R(G)$ [43].

Common Group Extraction. We next optimize the computation of $R_1(G) \cdot R_2(G)$ for two RPQs R_1 and R_2, by reducing redundant computation in the corresponding Boolean matrix multiplication. Since multiple source vertices in $R_1(G)$ may share common target vertices, computing the matrix product may repeatedly access the same rows of M_{R_2}. To avoid such redundancy, we group such source vertices.

We formulate this group task as the *maximum revenue problem*, denoted by MRP. Given $R_1(G)$, let $\mathcal{S}$ be the set of all source vertices in $R_1(G)$, it is to partition $\mathcal{S}$ into subsets (*i.e.,* groups) $P_1, \ldots, P_L$ to maximize the total revenue, which is defined as follows. For each subset P_i ($i \in [1, L]$), Let C_{P_i} denote the set of common target vertices reachable from all source vertices in P_i, *i.e.,* $C_{P_i} = \cap_{s \in P_i} \{v_t \mid (s, v_t) \in R(G)\}$. The revenue of P_i is defined as $(|P_i| - 1)|C_{P_i}|$, and the total revenue of the partition $\mathcal{P}$ is $\sum_{i=1}^{|\mathcal{P}|}(|P_i| - 1)|C_{P_i}|$.

Intuitively, when computing $R_1(G) \cdot R_2(G)$, we only need to retrieve and process the vertex pairs in $R_2(G)$ originating from C_{P_i} only once for each group P_i. We then take the Cartesian product of P_i and the resulting target vertices to generate the corresponding query results. Without such grouping, these vertex pairs in $R_2(G)$ would be repeatedly accessed, once for each source vertex in P_i. We develop an algorithm to identify such common groups in the next section (see Section III-B).

Remark. Biclique-based representation has been used in matrix product [15]. Given a two Boolean matrices A and B this approach first represents A as a graph G_A to identify bicliques (*i.e.,* complete bipartite subgraphs), that enable shared computation. It then decomposes A into two matrix componets A_1 and A_2, representing the edges covered by these bicliques and the remining edges in G_A, respectively. Similarly, matrix B is decomposed into B_1 and B_2. Then the matrix product $A \times B$ can be computed by $A_1 \times B_1 + A_1 \times B_2 + A_2 \times B_1 + A_2 \times B_2$, where $A_1 \times B_1$, $A_1 \times B_2$ and $A_2 \times B_1$ can be accelerated by sharing computation since both A_1 and B_1 represent sets of bicliques. Different from [15], we only partition the rows of A into groups and define the revenues of partitions to measure their quality. Based on the revenues, we formulate the maximum revenue problem, establish its complexity, and develop a lightweight heuristic algorithm to solve it. As shown in Section V, our heuristic algorithm is better suited to RPQC evaluation due to its lightweight design.

B. Algorithm for Common Group Extraction

We first study the complexity of MRP. We consider its decision version, denoted by DMRP, which is to decide, given $R(G)$ and an integer K, whether there exists a partition $\mathcal{P}$ of the source vertices of $R(G)$ such the total revenue is at least K, *i.e.,* $\sum_{i=1}^{|\mathcal{P}|}(|P_i| - 1)|C_{P_i}| \geq K$.

We first show that DMRP is intractable.

Theorem 1: *DMRP is* NP*-complete.* □

Proof Sketch: We start with the upper bound.

Upper bound. We develop an NP algorithm for DMRP: (1) guess a partition $\mathcal{P} = \{P_1, \ldots, P_L\}$ of source vertices in $R(G)$; and (2) check whether the total revenue is at least K; if so, return true. The algorithm is in NP since verification in step (2) is in PTIME. Thus, DMRP is in NP.

Lower bound. We prove the NP-hardness by reduction from the Linear Exact Cover by 3-Sets (LX3C) problem [44], [45], which is to decide, given a set $U = \{1, \ldots, 3n\}$ and a collection $\mathcal{S}$ of subsets $S_1, \ldots, S_k$ of U such that $|S_i| = 3$ ($i \in [1, k]$) and any two numbers $n_i, n_j \in U$ appear in at most one subset in $\mathcal{S}$, whether there exists an exact cover of U from $\mathcal{S}$, *i.e.,* a collection $\mathcal{S}' \subseteq \mathcal{S}$ such that $\mathcal{S}' = \{S'_1, \ldots, S'_n\}$, $U = S'_1 \cup \ldots \cup S'_n$, $S'_i \cap S'_j \neq \emptyset$ for each $i, j \in [1, n]$ with $i \neq j$. Given U and $\mathcal{S}$, we define (1) a set G_b of vertex pairs such that it consists of $3n$ source vertices, one for each element in U, and k target vertices, one for each set in $\mathcal{S}$; and if element i in U appears in subset S_j, there is a vertex pair (u_i, v_j) in G_b. (2) Number K is defined as $K = 2n$. We can show that there is a partition of source vertices in G_b whose total revenue is at least K if and only if there is an exact cover of U from $\mathcal{S}$. □

Heuristic algorithm for MRP. Since optimal grouping is NP-hard, we use a support-guided inverted-index method to group source vertices that share common target vertices.

Overview. Recall that the revenue of a partition $\mathcal{P}$ is defined as $\sum_{i=1}^{|\mathcal{P}|}(|P_i| - 1)|C_{P_i}|$, where each C_{P_i} is the set of common target vertices reachable from all source vertices in group P_i. To get a partition with the largest revenue, we should maximize both P_i and C_{P_i}. However, increasing the size of P_i may

Algorithm 1 GreedyGroup

Input: Given query results $R(G)$;
Output: A grouping of $R(G)$;

1: $\mathcal{T} := R(G)$; $\Delta = \emptyset$;
2: **while** $\mathcal{T} \neq \emptyset$ **do**
3: $m_a := \mathsf{MaxSupport}(\mathcal{T})$;
4: $S_a := \mathsf{GetSource}(\mathcal{T}, m_a)$;
5: $M_a := \mathsf{ExtractTarget}(\mathcal{T}, S_a)$;
6: $G_m := \mathsf{maxPrefix}(\mathcal{T}, S_a, M_a)$;
7: $\mathcal{T} := \mathcal{T} \setminus G_m$; $\Delta := \Delta \cup \{G_m\}$;
8: **return** Δ;

reduce the number of common target vertices in C_{P_i}, making it difficult to obtain a group with both a large $|P_i|$ and a large $|C_{P_i}$. To handle this, we adopt a greedy strategy.

Before presenting the algorithm, we first define some notations. Given a source vertex s, let $M(s)$ denote the set of target vertices t such that $(s,t) \in R(G)$. For each target vertex m, We build an inverted index $\mathsf{source}(m) = \{s' \mid m \in M(s')\}$, and define its support is defined as $\mathsf{support}(m) = |\mathsf{source}(m)|$.

Given the query result $R(G)$, we group source vertices by considering the target vertex with the maximum support in $R(G)$. Intuitively, let m be a target vertex with the maximum support in $R(G)$. The revenue of the group $\{(v,m) \mid (v,m) \in R(G)\}$ is at least $\max_{(u,v)\in R(G)} |\mathsf{source}(v)|$, which provides a lower bound for the revenue of each group.

Algorithm. We present the GreedyGroup procedure in Algorithm 1. Given a query result $R(G)$, Algorithm GreedyGroup iteractively groups source vertices by considering the target vertex with the maximum support (lines 2-7). Let $\mathcal{T}$ be the set of remaining vertex pairs, initially set to $R(G)$, and Δ the collection of generated groups, initially empty (line 1). In each iteration, the algorithm (1) selects the target vertex m_a with the maximum support in $\mathcal{T}$ (line 3); (2) fetches the set S_a of source vertices reaching m_a (line 4); (3) extracts target vertices from $\mathcal{T}$ that are reachable from the source vertices in S_a and gets M_a, which is used to compute the revenue (line 5); and (4) deduces a group by the set of vertices pairs with the maximum revenue, using S_a and M_a (line 6). More specifically, (a) it first sorts the vertices in M_a by decreasing support, and let $m_1, \ldots, m_K$ be the ordered vertices; (b) for each prefix $X_l = \{m_1, \ldots, m_l\}$, it collects the set G_l of source vertices reaching all target vertices in X_l, *i.e.,* $G_l = \bigcap_{i=1}^{l} \mathsf{source}(m_i)$; and (c) it then computes the revenue of each G_l, and selects the set G_m with the largest revenue. This process repeats until all vertex pairs in $\mathcal{T}$ are processed.

Example 6: Consider result $R(G)$ for $R = \mathsf{parent}$ on the graph in Figure 1. It consists of vertex pairs (u,v) such that (u, parent, v) is an edge in the graph. GreedyGroup extracts source-vertex groups from $R(G)$ as follows.

(1) It first selects a target vertex with the maximum support, *i.e.,* the maximum number of incoming edges labeled parent. Then JohnD.RockefellerJr. is selected as m_a.

(2) It then collects the source vertices of m_a, *i.e.,*David Rockefeller and JohnD. RockefellerIII, from which it further retrieves their target vertices in $R(G)$, *i.e.,*JohnD. Rockefeller Jr. and Abby Aldrich Rockefeller.

(3) Next it sorts these target vertices by decreasing support, resulting in JohnD. RockefellerJr. and Abby Aldrich Rockefeller, both with support of two.

(4) After that it computes the revenue for each prefix. For the prefix {John D. Rockefeller Jr. }, the revenue is $(2-1)\times 1=1$; and for the prefix {John D. Rockefeller Jr., Abby Aldrich Rockefeller }, the revenue is $(2-1)\times 2 = 2$. Thus, the group with the maximum revenue is $G_m = \{\mathsf{DavidRockefeller}, \mathsf{JohnD.Rockefeller\ III}\}$, since their common targets are {John D. Rockefeller Jr., Abby Aldrich Rockefeller }, and their revenue is 2.

Algorithm GreedyGroup repeats these steps until all source vertices in $R(G)$ are partitioned into disjoint groups. □

Properties. We next present some properties of GreedyGroup.

Lower bound. Given a query result $R(G)$, the revenue for the grouping returned by Algorithm GreedyGroup is at least $d_{\mathsf{tmax}} - 1$, where d_{tmax} is the maximum support for any target vertex, *i.e.,* $d_{\mathsf{tmax}} = \max_{(u,v)\in R(G)} |\mathsf{support}(v)|$. Actually, $d_{\mathsf{tmax}} - 1$ is the revenue of the first group, since (a) we start from the target vertex having the maximum support (*i.e.,* d_{tmax}); (b) we order the target vertices following the decreasing order of the support of target vertices; and (c) grouping source vertices using the prefix of the ordered target vertices, *i.e.,* the target vertex with the maximum support exists at the grouping.

For remaining groups, all target vertices may be connected to only one source vertice, leading to revenue 0. However, in practice, the generated groups have larger revenue than this lower bound, since $R(G)$ is dense, *i.e.,* few target vertices are connected to only one source vertices (see Section V).

Remark. Given partition $\mathcal{P} = \{P_1, \ldots, P_L\}$ of $R(G)$, we compute $R(G) \cdot R'(G)$ as follows: (1) first extract the set TV_f of target vertices of vertex pairs in P_i; (2) collect the set VP_s of vertex pairs in $R'(G)$, whose source vertices are in TV_f; and (3) compute $R(G) \cdot R'(G)$ by joining P_i and VP_s [46].

C. Evaluaion Algorithm for RPQCs

Putting these together, we present Algorithm RPQCEval to compute $Q(G)$ in Algorithm 2. Given a graph G, a RPQ R, a counting operator $\{m,n\}$ and a set $\mathcal{S}$ of initial vertices, RPQCEval computes a set of pairs (u,v) such that $u \in \mathcal{S}$ and $(u,v) \in R\{m,n\}(G)$. Then we invoke $\mathsf{RPQCEval}(G, Q, [1,1], V_G)$ to compute $Q(G)$ for a graph G and a RPQC Q, where V_G is the set of vertices in G. Algorithm RPQCEval works in the following steps:

(1) it first constructs the automaton $\mathcal{A}$ of R (line 1), and collects initial states $\mathcal{I}$ formed by the initial state q of $\mathcal{A}$ and vertices in $\mathcal{S}$ (line 2). We also use $\mathcal{R}$ to store the visited states to avoid recomputation.

(2) Then it travels automaton $\mathcal{A}$ to compute $R(G)$ (lines 3-13);

Algorithm 2 RPQCEval

Input: A graph G, a RPQ R, a counting operator $\{m, n\}$, and a set $\mathcal{S}$ of initial vertices;
Output: A set T of pairs (u, v) such that $u \in \mathcal{S}$ and $(u, v) \in R\{m, n\}(G)$;

1: $\mathcal{A} := \mathsf{ConstructAuto}(R)$;
2: $\mathcal{R} := \mathcal{I} := \mathsf{ComputeInit}(\mathcal{A}, \mathcal{S})$;
3: **while** $\mathcal{I} \neq \emptyset$ **do**
4: $(v_I, q, v) \leftarrow \mathrm{pop}(\mathcal{I})$;
5: **for all** $\delta((q, v), A) = (q_1, v_1)$ in $\mathcal{A}$ **do**
6: **if** A is a label in Σ and (q_1, v_1) is not in $\mathcal{R}$ **then**
7: $\mathsf{PR} := \{(v_I, q_1, v_1)\}$;
8: **else if** A is a symbol $C_{R\{m,n\}}$ **then**
9: $\mathsf{PR} := \mathsf{RPQCEval}(G, R, \{m, n\}, \{v\})$;
10: extend $\mathcal{I}$ and $\mathcal{R}$ with PR;
11: **if** $m > 1$ **then**
12: **for all** $(v_I, q_f, v_f) \in \mathsf{PR}$ with $(q_I, v_f) \notin \mathcal{R}$ **do**
13: extend $\mathcal{I}$ and $\mathcal{R}$ with (v_I, q_I, v_f) for q_I;
14: $k_{\max} := \mathsf{GetBound}(m, n)$; $i := 2$;
15: $\mathcal{F}^1 := \mathsf{Construct}(\mathcal{R})$; $\mathcal{V}^1 := \{\varepsilon\}$;
16: **while** $i \leq k_{\max}$ **do**
17: $\mathcal{C}^i := \mathsf{ExtractCommon}(\mathcal{F}^{i-1})$;
18: $\mathcal{F}^i := \mathsf{Product}(\mathcal{F}^{i-1}, \mathcal{F}^{i-1}, \mathcal{C}^i)$;
19: $\mathsf{TV}^i := \mathsf{Product}(\mathcal{V}^{i-1}, \mathcal{F}^{i-1})$; $\mathcal{V}^i := \mathcal{V}^{i-1} \cup \mathsf{TV}^i$;
20: $i := i + 1$;
21: $\mathsf{F} := \mathsf{Compose}(\overline{\mathcal{F}}, \overline{\mathcal{V}}, \{m, n\})$;
22: **return** F.

more specifically, for each initial state (q, v) in $\mathcal{I}$, Algorithm RPQCEval collects all transitions of (q, v) in $\mathcal{A}$. Given a transition $\delta((q, v), A) = (q_1, v_1)$ in $\mathcal{A}$, (a) if A is a symbol in Σ, *i.e.,* A is not a counting operator, then we directly store the reached state (q_1, v_1); (b) otherwise, if A is counting operator $R_1\{m, n\}$, and it recursively invokes RPQCEval to compute $R_1\{m, n\}(G)$ from vertex v only, *i.e.,* it computes the necessary results only (lines 6-9). After that, it extends $\mathcal{I}$ and $\mathcal{R}$ with the newly reached states in PR (line 10), *i.e.,* for each pair $(q, v) \in$. When $m > 1$, *i.e.,* RPQCEval collects all results in $R(G)$ to compute $R\{m, n\}$, it extends $\mathcal{I}$ with all newly added states in $\mathcal{R}$ and incrementally computes $R(G)$ (line 13).

(3) When it reaches a transition labeled $C_{R\{m,n\}}$, it first exploits the common group extraction and then applies the recursive doubling strategy to compute $R\{m, n\}(G)$ (lines 14-21). It first determines the upper bounds $k_{\max}$ for auxiliary RPQs F^i and V^i (line 14); when n is ∞, the query is rewritten as $R\{m\} \cdot R\{0, \infty\}$, and $k_{\max}$ is determined by m; and $R^*(G)$ is determined by computing the transitive closure of $R(G)$ [43]. It then extracts the common groups in $R(G)$ (line 17), and then exploits the recursive doubling strategy to compute $R\{m, n\}(G)$ using these common groups (lines 18-19).

(4) After that, it composes the result $R\{m, n\}(G)$ based on results F^i and V^i and the binary encoding of m and n, and return the composed relations (line 22).

Example 7: Given the graph G in Figure 1 and RPQC $Q' = \mathsf{parent}\{2, 4\} \cdot \mathsf{birthPlace}$ from Example 2, Algorithm RPQCEval computes $Q'(G)$ as follows.

(1) It first constructs automaton $\mathcal{A}_{Q_{\mathsf{cf}}}$ of $Q_{\mathsf{cf}} = C_{\mathsf{parent}\{2,4\}} \cdot \mathsf{birthPlace}$. Since the first transition of $\mathcal{A}_{Q_{\mathsf{cf}}}$ is $C_{\mathsf{parent}\{2,4\}}$, RPQCEval is recursively called to compute $\mathsf{parent}\{2, 4\}(G)$.

(3) During the recursion, it first computes $\mathsf{parent}(G)$; *e.g.,* from David Rockefeller, it reaches two vertices, *i.e.,* John D. Rockefeller, Jr. and Abby Aldrich Rockefeller.

(4) After that it calls GreedyGroup to group source vertices of $\mathsf{parent}(G)$, and computes $\mathsf{parent}\{2, 4\}(G)$ by applying the recursive doubling strategy.

(5) After returning $\mathsf{parent}\{2, 4\}(G)$, it follows the subsequent birthPlace transition of $\mathcal{A}_{Q_{\mathsf{cf}}}$ and reaches an accepting state. It records the final results using initial vertices and reached vertices, yielding four endpoint pairs for David Rockefeller and 13 distinct endpoint pairs over all vertices in graph G. □

Complexity. Algorithm RPQCEval takes $O(|Q||V|^3)$ time. Indeed, (1) constructing the automaton takes $O(|Q|)$ time; (2) traveling automaton takes $O(|Q||V|(|V| + |E|))$ time, since it travels at most all states in the automaton one for each starting vertex in G; and (3) Computing the counting operators takes $O(|Q||G|^3)$ time, since (a) Algorithm Evaluation is called for each counting operator and there exists at most $|Q|$ many counting operators; (b) the recursive doubling strategy takes $O(|V|^3(\log n + \log m)$ for each counting operator $R\{m, n\}$, since we exploit incremental computation and share the computation across different starting vertices. Here, $\log n + \log m$ is bounded by $|Q|$, since $|Q|$ is the size of the query Q, which is the sum of the number of symbols, the numbers of operators and the binary encoding of the counting operators in Q [42].

IV. Distributed Algorithm for RPQC

In this section, we develop a distributed Algorithm RPQCDEval to compute $Q(G)$ for processing large-scale graphs. Before presenting the algorithm, we first introduce a notation to evaluate the effectiveness of distributed algorithms.

Parallel scalability. Given a RPQ Q and a graph G, let $T(Q, G)$ be the worst-case runtime of Evaluate to compute $Q(G)$. We say that a distributed algorithm $\mathcal{A}_p$ is *parallel scalable relative to* Evaluate if for any RPQ Q and graph G, its runtime to compute $Q(G)$ using n workers (*i.e.,* processors) is $T(G, Q, n) = O(\frac{T(Q,G)}{n})$. Intuitively, this means that algorithm $\mathcal{A}_p$ can reduce the runtime of Evaluate when more processors are used, and can process large-scale graphs.

Challenges. It is challenging to achieve parallel scalability. (1) We need to evenly partition the computation across all processors. However, to compute the results of counting operators, we conduct matrix multiplication. How can we evenly partition such computation? (2) We also need to prevent the communication cost from dominating overall cost, which is the key challenging in distribute computation [47].

We address these challenges as follows.

(1) To balance computation, we partition G via edge-cut [48]; that is, G is partitioned into multiple fragments G_1=(V_1, E_1), . . . , G_n=(V_n, E_n) such that (a) $V = V_1 \cup \ldots \cup V_n$; (b) $V_i \cap V_j = \emptyset$ for $i \neq j$; and (c) each vertex in G_i has all its adjacent edges in G_i. We call a vertex in G_i as border vertex if v' is assigned to another fragment G_j (*i.e.,* $v' \in V_j$ with $i \neq j$) and has an edge to a vertex in V_i.

Let U_i denote the set of vertices assigned to fragment i, Then fragment i stores all results $(u, v) \in Q(G)$ with $u \in U_i$, denoted by $Q^i(G)$, during the computation. To compute matrix product, we further partition $Q^i(G)$ into multiple parts based on their target vertices, *i.e.,* $Q^i(G) = Q^{i,1}(G) \cup \cdots \cup Q^{i,n}(G)$, such that $Q^{i,j}(G)$ contains all pairs $(u, v) \in Q^i(G)$ with $v \in U_j$. These blocks are distributed for matrix computation.

We can verify that communication cost are evenly partitioned to compute matrix product. Given a matrix $Q_1(G)$, to compute $Q_1(G) \circ Q_1(G)$ a worker p_i sends $O(\frac{|V|}{n}|V|)$ message to other $n-1$ workers, since it only sends the results start from vertices in U_i, and receive $O(\frac{|V|}{n}|V|)$ messages with vertices in U_i as target vertices from other workers. Therefore, a worker sends and receives $O(\frac{|V|^2}{n})$ messages.

(2) We exploit two strategies to balance the computation. Different source vertices can reach different target vertices via the RPQC queries, and some vertices may reach a large number of vertices, leading to some workers overloaded while others remain idle. To handle this, (a) we first estimate the computation cost of each fragment, *e.g.,* the total number of source vertices in $R(G)$, before matrix multiplication; and (b) we then rebalance computation using the following strategies:

(i) before computing the matrix product, we evenly distributed the vertex pairs in $R(G)$ across all workers based on the hash values of the source vertex IDs [49].

(ii) during evaluating counting operators with the recursive doubling strategy, some source vertices may reach a large vertex set again, leading to overloaded workers; therefore, we exploit work stealing [50], [51] to move the matrix product of some vertices from overloaded workers to workers with smaller computation workload. The workload is estimated based on the total revenue of the grouping in each fragment.

(3) We exploit the sparse matrix compression like CSR or CSC to reduce the communication cost [52]. Indeed, the query result $R(G)$ is much smaller than the Cartesian product of vertices in G. Moreover, the query result $R(G)$ can be represented as a Boolean matrix, which can be further compressed; for example, AdaBool partitions the matrix into blocks and selects encoding strategies based on the local sparsity of each block [53].

Algorithm. Putting these together, we design Algorithm RPQCDEval to compute $Q(G)$ following the PIE program ρ based on fixpoint operator $\rho(P_1, \ldots, P_n)$ [54] as follows:

$$R_i^0 = \mathsf{PEval}(Q, G_i),$$
$$R_i^k = \mathsf{IncEval}(Q, G_i, R_i^{k-1}, M_i).$$

The algorithm first invokes PEval to evaluate Q on G_i; then it constructs message for further inspecting, and sends to other workers; after receiving message M_i, it calls IncEval to incrementally evaluate Q on G_i and local partial result R_i^{k-1}. The algorithms terminate when no message is generated.

Algorithm 3 PEval

Input: A graph G_i and a RPQ R;
Output: Set T of pairs (u, v) such that $(u, v) \in Q(G_i)$;

1: $\mathcal{A} := \mathsf{ConstructAuto}(Q)$;
2: $\mathcal{H}_{(G_i,\mathcal{A})} := \mathsf{ProductionA}(\mathcal{A}, \mathcal{G}_i)$;
3: $\mathsf{EF}_i := \mathsf{ExtEvaluate}(\mathcal{H}_{(G_i,\mathcal{A})}, V_{G_i})$;
4: build messages M_{ij} from EF_i and send M_{ij} to worker p_j.

Algorithm 4 IncEval

Input: A graph G_i, a RPQ Q, partial result $\mathcal{R}_i$ and message $\mathcal{M}_i$;
Output: New set $\Delta\mathcal{R}_i$ of pairs $(u, v) \in Q(G)$;

1: $\Delta\mathcal{R}_i = \emptyset$; $M_i := \emptyset$;
2: **for all** message M in $\mathcal{M}_i$ **do**
3: **if** M is in the form of (R, v_I, q, v) **then**
4: $(\Delta\mathcal{R}_i, \Delta M_i) := \mathsf{TraveralAuto}(G_i, R, v_I, v, q)$;
5: **else if** M is in the form of $(R, v_I, \{m, n\}, c)$ **then**
6: $(\Delta\mathcal{R}_i, \Delta M_i) := \mathsf{IncProd}(\mathcal{R}_i, R, v_I, \{m, n\}, c)$;
7: $\mathcal{R}_i := \mathcal{R}_i \cup \Delta\mathcal{R}_i$; $M_i = M_i \cup \Delta M_i$;
8: send M_{ij} to worker p_j.

Before presenting the algorithms, we first define the message, which has the following two forms:

(1) (R, v_I, q, v) indicating that (a) PEval or IncEval is to compute $R(G)$ from vertex v_I by traveling the automaton of R and G; (b) PEval or IncEval has match v to state q; and (c) IncEval matches the next state in automaton; and

(2) $(R, v_I, \{m, n\}, c, V_t)$ indicating that (a) Algorithm PEval or IncEval has computed the result $R(G)$; (b) $R^c(G)$ has been computed, and reaches vertices in V_t; and (c) IncEval needs to compute $R^{2c}(G)$ from vertices in V_t.

Before sending such message, we first compress them. To this end, we first group these messages based on the RPQC R and counting operators $\{m, n\}$, and then compress using the sparse matrix compression like CSR or CSC [52].

Details of RPQCDEval. We present the PEval and IncEval of RPQCDEval in Algorithm 3 and 4, respectively.

Algorithm PEval. Given a graph G_i and a RPQ Q, PEval computes $Q(G_i)$ as Algorithm Evaluate, except that (1) it does not compute counting operator $R\{m, n\}(G)$, since $R(G)$ may not have been received from other workers; and (2) it generates messages in the above form based on $Q(G_i)$. More specifically, it generates messages of the form (R, v_I, q, v) when v is a vertex does not belong to Q_i, *i.e.,* the computation continues from graph G_j having v.

Algorithm IncEval. Given a graph G_i, a RPQ Q, partial result $\mathcal{R}_i$ and message $\mathcal{M}_i$, it computes a new set $\Delta\mathcal{R}_i$ of pairs $(u, v) \in Q(G)$ based on the received messages in $\mathcal{M}_i$:

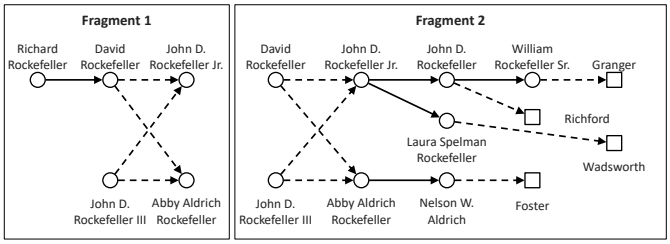


Fig. 3: The DBpedia example subgraph partitioned across two workers for evaluating parent$\{2,4\}$/birthPlace.

(1) If the message is in the form of (R, v_I, q, v), then it invokes Algorithm TraveralAuto to travel the automaton of R from vertex v and state q, and generates message as PEval: (a) when it reaches a vertex that is not in G_i, it generates messages of the form (R, v_I, q, v) (line 4); and (b) if it reaches a symbol $C_{R_1\{m,n\}}$, it first computes the results $R_i(G)$, and then generates a message $(R_1, v_I, \{m, n\}, 1)$ to match the counting operators using the recursive doubling strategy; and

(2) If the message is $(R, v_I, \{m, n\}, c, V_t)$, it invokes IncProd to continue the matches. Based on c, consider the followings.

(a) If $0 < c \leq n$, *i.e.,* the result $R\{c\}(G_i)$ has been computed, then IncProd computes $R\{2c\}(G_i)$ using both local partial results $\mathcal{R}_i$ and received partial matches $R\{c\}(G_i)$, and generates message to the workers to continue the computation; and

(b) If $c > n$, *i.e.,* all results $R(G_i) \ldots, R\{c\}(G_i)$ are computed, IncProd computes $R\{m, n\}(G)$ using the recursive doubling strategy, and generates message for further computation based on the hierarchical representation of RPQCs.

Example 8: Consider query $Q' =$ parent$\{2,4\} \cdot$ birthPlace in Example 2 and the graph in Figure 1. Assume that the graph is partitioned into two fragments, as shown in Figure 3. Dashed parent arrows denote cross-fragment edges, shown in both fragments.

Algorithm RPQCDEval computes $Q(G)$ as follows.

(1) At each fragment, RPQCDEval invokes PEval to compute $Q(G_i)$ $(i \in [1, 2])$, and generates message for further processing; *e.g.,* at fragment f_1, it generates messages (parent, David Rockefeller, $[2, 4]$, 1, {John D. Rockefeller Jr., Abby Aldrich Rockefeller }) and (parent, Richard Rockefeller, [2, 4], 2, {John D. Rockefeller Jr., Abby Aldrich Rockefeller }), since the computation reaches the border vertices of f_1, and the computation of parent$\{2,4\}$ continues in the fragment having these border vertices, *i.e.,* fragment f_2.

(2) After receiving these messages, IncEval continues to compute parent$\{2,4\}(G)$; *e.g.,* given message (parent, David Rockefeller, [2, 4], 1, {John D. Rockefeller Jr., Abby Aldrich Rockefeller }), IncEval apply the recursive doubling strategy to compute parent$\{2,4\}(G)$ in fragment f_2, which returns four vertex pairs with David Rockefeller as source vertex. From the target vertices of these vertex pairs, IncEval continues to compute birthPlace(G) in fragment f_2, and reaches an accepting state, which returns vertex pairs in $Q(G)$. □

TABLE II: Dataset statistics

Dataset	$\|V\|$	$\|E\|$	$\|\Gamma\|$	Domain
DBpedia	5.526M	18.295M	661	Encyclopedic knowledge
YAGO2s	12.404M	115.889M	48	General knowledge
LDBC SNB SF0.3	0.908M	4.583M	23	Social network
LDBC SNB SF1	3.182M	17.256M	23	Social network
LDBC SNB SF3	9.282M	52.696M	23	Social network
LDBC SNB SF10	29.988M	176.623M	23	Social network

Parallel scalability. We next show that RPQCDEval is parallel scalable with respective to Evaluation. To this end, we show by induction on the structure of Q that both PEval and IncEval take $O(\frac{|Q||V|^3}{n})$, where n is the number of workers. We only sketch the proof for $Q_1\{m_1, n_1\}$; see [55] for detailed proof.

When Q is in the form of $Q_1\{m_1, n_1\}$, by the induction assumption, computing $Q_1(G)$ is in $O(\frac{|Q_1||V|^3}{n})$ time. Given $Q_1(G)$, we distribute $Q_1(G)$ across workers to compute $Q_1\{m_1, n_1\}(G)$ using the recursive doubling strategy. Observe that (a) the recursive doubling strategy takes at most $O(|Q|)$ rounds, one for each bit in binary encodes of m_1 and n_1; when n is ∞, we rewrite $Q_1\{m_1, n_1\}(G)$ into $R\{m\}(G) \cdot R^*(G)$ and compute $R^*(G)$ using transitive closure of $R(G)$ [43]; (b) in each round IncEval takes $O(\frac{|V|^3}{n})$ time to compute the matrix product for auxiliary RPQs F^i and V^i, and $O(\frac{|V|^3}{n})$ time to send messages to other workers or receive messages from other workers; and (d) it computes $Q\{m, n\}(G)$ using F^i and V^i, which takes $O(|Q| \cdot \frac{|V|^3}{n})$ time.

Remark. We can adopt asynchronous parallel [56] to accelerate computation. Observe that different workers may have different workload due to the graph partition. Then a worker invokes IncEval when it finishes the current computation and receives messages from other workers without synchronizing with other workers. This does not impact the correctness of the algorithm, since (a) the computation is monotonic, (b) the computations are triggered by messages and (c) there does not exist any dependency among messages [57].

V. Experimental Evaluation

Using real-life and synthetic graphs, we conduct experiments to evaluate (a) the efficiency and (b) parallel scalability of RPQCDEval. We also conduct (c) an ablation study and (d) a cost breakdown to assess the contribution of each component of RPQCEval to its overall performance.

Experimental Setting. We start with experimental settings.

Datasets. We used two real-life graphs: (1) DBpedia, a knowledge graph with 5.5M vertices and 18.3M edges extracted from Wikipedia [58]; and (2) YAGO2s with 12.4M vertices and 115.9M edges integrated from multiple sources, including Wikipedia, WordNet and GeoNames [59].

We generated synthetic datasets to evaluate scalability of proposed algorithms. We exploited LDBCSNB to simulate social-network interactions [60], and used factors 0.3, 1, 3, and 10 to yield SF0.3, SF1, SF3, and SF10 datasets, respectively. Datasets SF0.3 and SF10 are mainly used in the scalability test.

Table II summarizes the statistics of all datasets.

TABLE III: Counting RPQ workload categories.

Class	Queries	Template	Description
C1	Q1–Q3	$a\{k\}$	Single-label exact counting
C2	Q4–Q6	$(a_1 \cdot \ldots \cdot a_l)\{k\}$	Repeated sequence
C3	Q7–Q9	$(a_1 \mid \ldots \mid a_l)\{k\}$	Repeated alternative
C4	Q10–Q12	$A\{\ell, u\}$	Bounded-range counting
C5	Q13–Q15	$A\{\ell, \infty\}$	Lower-bounded unbounded counting
C6	Q16–Q18	$(A\{k_1\} \mid B\{k_2\})\{k_3\}$	Composite and nested counting

Query workload. Following [14], [61], we organize the query workload into six classes C1–C6, as summarized in Table III. More specifically, (a) C1–C3 cover exact counting with different path-body structures; (b) C4 covers bounded-range counting expressions $B\{m, n\}$; (c) C5 covers unbounded counting expressions $B\{m, \infty\}$; and (d) C6 covers composite and nested counting expressions involing concatenation and union.

We instantiate three queries for each category using query patterns from query logs, benchmarks, and prior studies. Specifically, queries for DBpedia are derived from LSQ query logs [62] and the property-path study by [63]; queries for YAGO2s are derived from prior graph-query studies [64], [65]; queries for LDBC SNB are derived from the benchmark [60].

Algorithms. We implemented the following in C++: (1) RPQC evaluation algorithm RPQCEval (Section III); (2) distributed algorithm RPQCDEval (Section IV); (3) $\mathsf{RPQCEval}_{\mathsf{ng}}$, a variant that does not group vertices in matrix multiplication; (4) $\mathsf{RPQCEval}_{\mathsf{nr}}$, a variant that does not use recursive doubling and directly computes exact-count relations for all repetitions in the bounded-ranges; (5) $\mathsf{RPQCEval}_{\mathsf{bc}}$, a variant that exploits a biclique-based method [15] to compute common groups; (6) $\mathsf{RPQCEval}_{\mathsf{si}}$, a variant that rewrites $R\{m, n\}$ as $R^m \cdot (\varepsilon | R)^{n-m}$, and computes results using binary encodes of m and $n - m$; and (7) $\mathsf{RPQCEval}_{\mathsf{ni}}$, a non-incremental variant that computes complete results for all source vertices of subexpressions, rather than only reached source vertices.

We compared proposed algorithms with the following: (8) ProductGraph(PG), an automata-based RPQ evaluator that traverses the product of the data graph and the query NFA [6], [32]; (9) Neo4j, a graph database supporting variable-length relationship patterns in Cypher [66]; (10) JenaARQ and Virtuoso, two RDF engines supporting SPARQL property paths [67], [68]; (11) RPQ-Matrix, a Boolean-matrix-based RPQ evaluator [33]; and (12) Ring-RPQ, which combines bit-parallel automaton simulation with the ring index [32].

For baselines that do not support counting operators, we rewrite $R\{m, n\}$ into equivalent queries $R \ldots R \cdot (\varepsilon + R) \ldots (\varepsilon + R)$. Such expansion does not degrade the performance of baselines, since the bounds for counting operators in the datasets are small, *e.g.,* the maximal bound is 40 in used RPQs, and after the expansion, the length is 497, which is much smaller than the sizes of graphs.

We do not include existing distributed RDF query engines, *e.g.,* AdPart [69], TriAD [70] and PRoST [71], as baselines because their implementations do not support certain RPQ operators in our datasets, like union and Kleene star.

Configuration. We run the experiments on a Linux server with 32 workers, each with an Intel Xeon Silver 4214 CPU at 2.2 GHz and 128 GB of RAM. We set a timeout of 3,600s for each run, and exclude runs terminated by out-of-memory (OOM) errors. We used METIS to partition graphs [72]. Each experiment was repeated five times, and the average is reported.

Experimental Results. We now report our findings.

Exp-1: Efficiency. We evaluate the efficiency of RPQCEval.

Query time. We compare RPQCEval with six baselines on the four main datasets, using 18 queries per dataset, and report the results in Figure 4(a–d). We find the following: (1) For each dataset, we report the minimum speedup across the six pairwise baseline comparisons. These speedups range from $6.75\times$ to $35.78\times$. These results demonstrate that RPQCEval effectively reduces redundant computation using optimization like common group extraction and recursive doubling. (2) The largest performance gap between RPQCEval and the baseline is observed on queries of classes C4 and C5 in Table III, which involves bounded-range counting; *e.g.,* the speedups range from $2.44\times$ to $33.52\times$ for queries in C4. This is because the recursive doubling and common group extraction can reduce redundant computation in bounded-range counting.

Workload robustness. We evaluate workload robustness using 90 additional counting RPQs per dataset, with 15 queries from each class, which are generated from random walks on graphs. We report speedups of RPQCEval over the six baselines in Figure 4(e). We found the following: RPQCEval outperforms PG by $71.11\times$, $2.28\times$, $2.05\times$, and $2.48\times$, and beats RPQ-Matrix by $7.64\times$, $2.52\times$, $2.60\times$, and $2.37\times$ on DBpedia, YAGO2s, SF1, and SF3, respectively. Similarly, speedup ranges are $2.39\times$–$22.40\times$, $2.38\times$–$178.44\times$, $2.50\times$–$189.85\times$, and $2.23\times$–$139.49\times$ over Ring-RPQ, Jena ARQ, Virtuoso, and Neo4j, respectively. RPQCEval achieves lower runtimes across diverse workloads, due to optimizations like recursive doubling and common group extraction.

Correctness. Using 360 queries in the robustness test, we evaluate the correctness of RPQCEval by comparing its result with those of baselines. We find that RPQCEval and all baselines return the same set of vertex pairs for every completed query.

Exp-2: Parameter Sensitivity. We evaluated the impact of the following parameters on the performance of RPQCEval: (a) the repetition bound k in RPQC, and (b) the nesting depth

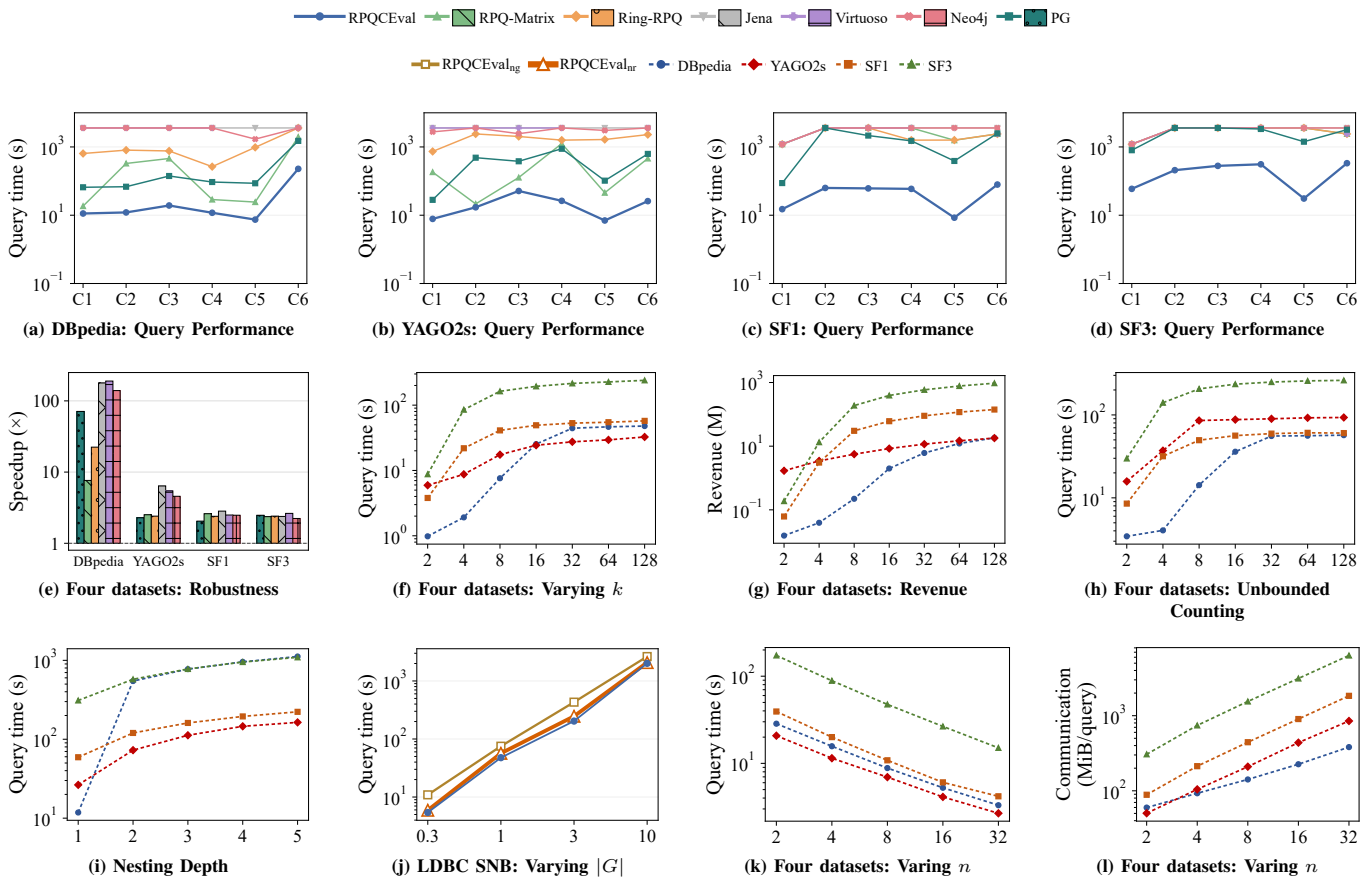


Fig. 4: Performance evaluation.

TABLE IV: Ablation results.

	DMCP				Recursive doubling			Incremental computation	
	Time (s)			Revenue (M)	Time (s)			Time (s)	
Dataset	RPQCEval$_{ng}$	RPQCEval$_{bc}$	RPQCEval	RPQCEval	RPQCEval$_{nr}$	RPQCEval$_{si}$	RPQCEval	RPQCEval$_{ni}$	RPQCEval
DBpedia	73.302	171.648	48.359	1.554451	548.635	144.827	129.448	606.591	499.310
YAGO2s	30.651	357.023	22.578	5.113613	93.279	58.646	36.272	228.171	26.941
SF1	75.126	583.267	47.656	67.768237	200.173	103.212	65.367	130.089	79.461
SF3	431.484	2560.070	203.648	431.050377	1000.206	507.469	332.003	559.146	397.538

d of counting operators in RPQC; *e.g.,* the nesting depth of $(a\{1,2\})\{2,3\}$ is 2. We select three workload queries per dataset for each sensitivity experiment.

Varying k. We varied k from 2 to 128. As shown in Figures 4(f)-(g), (1) the runtime of RPQCEval increases with k, as expected. Algorithm RPQCEval is $48.75\times$ slower on DBpedia when k varies from 2 to 128, since larger bounds require more matrix product. (2) Total revenue increases with k on all four datasets, as larger bounds involve more composition stages and provide further opportunities to reuse intermediate results. The revenue on YAGO2s increases only slightly when k gets larger, since target vertices of queries like $(\mathsf{sameAs} \mid \mathsf{subClassOf})\{k\}$ are concentrated on a small set of vertices, and increasing k only increases revenues modestly.

We also evaluated the impact of unbounded-ranges on the performance of RPQCEval. We evaluated queries of the form $R\{k,\infty\}$ by varying k from 2 to 128. As shown in Figure 4(h), (1) the runtime increases with k, as expected, since larger k requires more matrix product; (2) the runtime becomes stables when k increases beyond a certain threshold. From example, if k is larger than 32, the runtime of RPQCEval increases by only 2.4%, 3.7%, 1.5%, and 4.9% on DBpedia, YAGO2s, SF1, and SF3, respectively, when k increases from $k = 32$ to $k = 128$. This is because we exploit the incremental computation to evaluate $R\{k,\infty\}$, when k gets larger, fewer new vertices can be reached by RPQCEval, *i.e.,* RPQCEval becomes stable.

Varying nesting depth. We vary nesting depth d from 1 to 5 by recursively nesting the three C4 queries. As shown in Figure 4(i), increasing d slows down RPQCEval. When d varies from 1 to 5, RPQCEval is $94.71\times$, $6.20\times$, $3.76\times$, and $3.52\times$

slower on DBpedia, YAGO2s, SF1, and SF3, respectively, since larger d results in more rounds of matrix product.

Varying $|G|$. We evaluate the scalability of RPQCEval and its two variants, RPQCEval$_{\mathsf{ng}}$ and RPQCEval$_{\mathsf{nr}}$, using LDBC SNB graphs with scale factors 0.3, 1, 3, and 10. Figure 4(j) reports query times over the same 18 queries, with timeouts assigned 3600s. We found the following: RPQCEval becomes slower when the graph size increases, as expected. From SF0.3 to SF10, its query time grows to 371.52 times its initial value. (2) RPQCEval is more efficient than its variants when processing large-scale graphs. On SF10 with 29.998M vertices and 176.623M edges, RPQCEval$_{\mathsf{ng}}$, RPQCEval$_{\mathsf{nr}}$, and RPQCEval encounter 13, 6 and 4 timeouts, respectively. Since only three of the 18 queries contain bounded-range expressions, the performance of RPQCEval$_{\mathsf{nr}}$ and RPQCEval are similar.

Varying n. We evaluate the parallel scalability of RPQCDEval by varying the number of workers from 2 to 32. As shown in Figure 4(k)-(l), (1) RPQCDEval gets faster when the number of workers increases, as expected. It becomes $11.50\times$, $9.39\times$, $8.61\times$, and $7.76\times$ faster on SF3, SF1, DBpedia, and YAGO2s, respectively, when n varies from 2 to 32. (2) RPQCDEval is parallel scalable relative to sequential algorithm RPQCEval. When $n = 32$, RPQCDEval is $14.61\times$, $8.44\times$, $11.38\times$, and $13.53\times$ faster than RPQCEval on DBpedia, YAGO2s, SF1, and SF3, respectively, which confirms analysis in Section IV.

Exp-3: Ablation study. We compare RPQCEval with its variants RPQCEval$_{\mathsf{ng}}$ and RPQCEval$_{\mathsf{nr}}$, which do not use common group extraction and recursive doubling strategy, respectively. We also compare with RPQCEval$_{\mathsf{bc}}$, RPQCEval$_{\mathsf{si}}$, and RPQCEval$_{\mathsf{ni}}$ to evaluate Biclique-based grouping, standard interval evaluation, and full relation computation, respectively.

DMCP. We first evaluated the effect of common group extraction using 18 queries per dataset. As shown in Table IV, (1) DMCP effectively reduces runtime. Algorithm RPQCEval outperforms RPQCEval$_{\mathsf{ng}}$ by $1.52\times$, $1.36\times$, $1.58\times$ and $2.12\times$ on DBpedia, YAGO2s, SF1 and SF3, respectively. This is because common groups can share computations in the same group, which reduce redundant computation. (2) Algorithm GreedyGroup effectively identifies common groups. The revenue of the grouping generated via GreedyGroup are 35.17%, 46.58%, 71.93%, and 72.25% of the matrix-operation work on DBpedia, YAGO2s, SF1, and SF3, respectively. (3) Reducing matrix-operation improves performance; *e.g.,* on SF3, the revenue of groups generated via GreedyGroup is 72.25%, and RPQCEval outperforms RPQCEval$_{\mathsf{ng}}$ by $2.12\times$, the largest improvements among four datasets. (4) RPQCEval achieves speedups of $3.55\times$, $15.81\times$, $12.24\times$ and $12.57\times$ over RPQCEval$_{\mathsf{bc}}$ on DBpedia, YAGO2s, SF1 and SF3, respectively, since the complexity of GreedyGroup is lower than that of the grouping strategy using bicliques [15].

Recursive doubling strategy. We evaluate the impact of recursive doubling on the performance of RPQCEval using nine bounded-range queries per dataset. As shown in Table IV, (1) the strategy is effective. RPQCEval is $4.24\times$, $2.57\times$, $3.06\times$, and $3.01\times$ faster than RPQCEval$_{\mathsf{nr}}$ on DBpedia, YAGO2s, SF1, and SF3, respectively. This is because the strategy reuses intermediate results to compose results for bounded-range queries, which reduces redundant computation. (2) RPQCEval is $1.12\times$, $1.62\times$, $1.58\times$, and $1.53\times$ faster than RPQCEval$_{\mathsf{si}}$ on DBpedia, YAGO2s, SF1, and SF3, respectively, since $(\varepsilon|R)(G)$ has more non-zero elements than $R(G)$, introducing redundancy in matrix production.

Incremental computation. We next evaluate the impact of incremental computation on the performance of RPQCEval. As shown in Table IV, RPQCEval achieves speedups of $1.21\times$, $8.47\times$, $1.64\times$, and $1.41\times$ over RPQCEval$_{\mathsf{ni}}$ on DBpedia, YAGO2s, SF1, and SF3, respectively, since incremental computation avoid matrix product for unreachable vertices.

TABLE V: Cost breakdown.

Dataset	AC	AT	CG	MC
DBpedia	0.0002%	11.7107%	3.4623%	84.8268%
YAGO2s	0.0006%	11.7382%	11.8539%	76.4073%
SF1	0.0003%	4.2377%	8.8013%	86.9606%
SF3	0.0001%	2.8880%	10.7080%	86.4039%

Exp-4: Cost breakdown. We divide the runtime of RPQCEval into four phases: (a) automaton construction (AC); (b) automaton traversal (AT); (c) common group extraction (CG); and (d) matrix computation for counting operators (MC). As shown in Table V, (1) MC accounts for 76.41%–86.96% of runtime and dominates the overall cost, as expected, since such computation cannot exploit the locality of graphs to accelerate computation. (2) AT accounts for 2.89%–11.74%, since RPQCEval exploits automata to compute query results, and the numbers in the counting operators are small. (3) Algorithm GreedyGroup is efficient, *i.e.,* CG accounts for only 3.46%–11.85%. (4) The runtime of AC is negligible, since queries are small and automata are constructed only once.

Summary. We find the following. (1) RPQCEval handles large graphs efficiently. It takes 203.648s to process graph SF3 with 9.282M vertices and 52.696M edges. (2) The optimizations are effective; recursive doubling strategy and DMCP achieve speedups of $2.57\times$–$4.24\times$ and $1.36\times$–$2.12\times$, respectively. (3) RPQCDEval is parallel scalable with respective to RPQCEval. RPQCDEval achieves speedups of $7.76\times$–$11.50\times$ when the number of workers varies from 2 to 32.

VI. Conclusion

We proposed Algorithm RPQCEval to evaluate regular path queries with counting operators. To accelerate computation, we develop two optimization strategies, namely common group extraction and recursive doubling strategy. We develop a distributed algorithm that is parallel scalable with respective to the sequential algorithm RPQCEval. Experimental results show the effectiveness and efficiency of proposed algorithms.

One topic for future work is to support more expressive regular path queries like conjunctive regular path queries. Another topic is to exploit GPU to accelerate the computation.

ACKNOWLEDGMENT

Generative AI tools were used only for limited language refinement and grammar enhancement during manuscript preparation. The proposed methods, algorithm design, theoretical formulations, experimental setup, implementation, and empirical analyses were all designed, conducted, and verified by the authors. No generative AI tools were used to generate experimental data, evaluation results, or source code.

REFERENCES

[1] A. Hogan, E. Blomqvist, M. Cochez, C. D'amato, G. D. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[2] Q. D. Truong, Q. B. Truong, and T. Dkaki, "Graph methods for social network analysis," in *ICTCC*, 2016, pp. 276–286.

[3] K. Erciyes, "Graph-theoretical analysis of biological networks: a survey," *Computation*, vol. 11, no. 10, p. 188, 2023.

[4] A. Deutsch, N. Francis, A. Green, K. W. Hare, B. Li, L. Libkin, T. Lindaaker, V. Marsault, W. Martens, J. Michels, F. Murlak, S. Plantikow, P. Selmer, O. van Rest, H. Voigt, D. Vrgoc, M. Wu, and F. Zemke, "Graph pattern matching in GQL and SQL/PGQ," in *SIGMOD*, 2022, pp. 2246–2258.

[5] ISO, "Information technology — database languages — gql," Tech. Rep. ISO/IEC 39075:2024, 4 2024.

[6] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter, and D. Vrgoč, "Foundations of modern query languages for graph databases," *ACM Computing Surveys*, vol. 50, no. 5, pp. 68:1–68:40, 2017.

[7] S. Harris, A. Seaborne, and E. Prud'hommeaux, "SPARQL 1.1 query language," W3C Recommendation, 2013. [Online]. Available: https://www.w3.org/TR/sparql11-query/

[8] Y. Wang, J. Chu, Y. Yuan, Y. Gu, H. Ji, and H. Zhang, "Label constrained reachability queries on time dependent graphs," in *ICDE*, 2024, pp. 3244–3256.

[9] E. V. Kostylev, J. L. Reutter, M. Romero, and D. Vrgoč, "SPARQL with property paths," in *ISWC*, vol. 9366, 2015, pp. 3–18.

[10] K. Casel and M. L. Schmid, "Fine-grained complexity of regular path queries," *Logical Methods in Computer Science*, vol. 19, 2023.

[11] W. Martens, M. Niewerth, and T. Popp, "A trichotomy for regular trail queries," *Logical Methods in Computer Science*, vol. 19, 2023.

[12] A. Le Glaunec, L. Kong, and K. Mamouras, "Regular expression matching using bit vector automata," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, pp. 492–521, 2023.

[13] L. Turoňová, L. Holík, O. Lengál, O. Saarikivi, M. Veanes, and T. Vojnar, "Regex matching with counting-set automata," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, pp. 218:1–218:30, 2020.

[14] L. Holík, J. Síč, L. Turoňová, and T. Vojnar, "Fast matching of regular patterns with synchronizing counting," in *FoSSaCS*, vol. 13992, 2023, pp. 392–412.

[15] N. Araya, J. Zagal, C. Hernández, and G. Navarro, "Compressed boolean matrix multiplication using bicliques," 2026.

[16] J. L. Reutter, M. Romero, and M. Y. Vardi, "Regular queries on graph databases," *Theory of Computing Systems*, vol. 61, no. 1, 2017.

[17] D. Calvanese, T. Eiter, and M. Ortiz, "Regular path queries in expressive description logics with nominals," in *IJCAI*, 2009, pp. 714–720.

[18] A. Deutsch and V. Tannen, "Optimization properties for classes of conjunctive regular path queries," in *DBPL*, 2001, pp. 21–39.

[19] M. Abo Khamis, A.-M. Hurjui, A. Kara, D. Olteanu, D. Suciu, and Z. Tian, "Output-sensitive evaluation of acyclic conjunctive regular path queries," in *ICDT*, 2026, pp. 18:1–18:20.

[20] D. Figueira, R. Morvan, and M. Romero, "Minimizing conjunctive regular path queries," *Proc. ACM Manag. Data*, vol. 3, no. 2, pp. 1–25, 2025.

[21] A. Amarilli, W. Gatterbauer, N. Makhija, and M. Monet, "Resilience for regular path queries: Towards a complexity classification," *Proc. ACM Manag. Data*, vol. 3, no. 2, pp. 1–18, 2025.

[22] G. Grahne and A. Thomo, "Regular path queries under approximate semantics," *Ann. Math. Artif. Intell.*, vol. 46, no. 1, pp. 165–190, 2006.

[23] X. Gou, X. Ye, L. Zou, and J. X. Yu, "LM-SRPQ: efficiently answering regular path query in streaming graphs," *Proc. VLDB Endow.*, vol. 17, no. 5, pp. 1047–1059, 2024.

[24] S. Zhang, K. Zhang, Z. He, Y. Jing, Z. Zhao, and X. S. Wang, "DRPQ: distributed evaluation of regular path queries on streaming graphs," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 317:1–317:27, 2025.

[25] A. Pacaci, A. Bonifati, and M. T. Özsu, "Regular path query evaluation on streaming graphs," in *SIGMOD*, 2020, pp. 1415–1430.

[26] M. Arenas, P. Bahamondes, A. Aghasadeghi, and J. Stoyanovich, "Temporal regular path queries," in *ICDE*, 2022, pp. 2412–2425.

[27] I. Na, Y.-S. Moon, I. Yi, K.-Y. Whang, and S. J. Hyun, "Regular path query evaluation sharing a reduced transitive closure based on graph reduction," in *ICDE*, 2022, pp. 1675–1686.

[28] T. Ren, L. Yang, X. Zhang, and Z. Feng, "Kleene closure property path query optimization based on node clustered index," in *WWW*, 2026, pp. 4160–4171.

[29] T. Ren, X. Zhang, and Z. Feng, "Optimization of kleene closure regular path query on large RDF graphs," *Comput. J.*, vol. 68, no. 11, pp. 1595–1609, 2025.

[30] D. Calvanese, G. D. Giacomo, M. Lenzerini, and M. Y. Vardi, "Answering regular path queries using views," in *ICDE*, 2000, pp. 389–398.

[31] Y. Pang, L. Zou, J. X. Yu, and L. Yang, "Materialized view selection & view-based query planning for regular path queries," *Proc. ACM Manag. Data*, vol. 2, no. 3, p. 152, 2024.

[32] D. Arroyuelo, A. Hogan, G. Navarro, and J. Rojas-Ledesma, "Time-and space-efficient regular path queries," in *ICDE*, 2022, pp. 3091–3105.

[33] D. Arroyuelo, A. Gómez-Brandón, and G. Navarro, "Evaluating regular path queries on compressed adjacency matrices," *VLDB J.*, vol. 34, no. 1, p. 2, 2025.

[34] R. Azimov and S. Grigorev, "Context-free path querying by matrix multiplication," *GRADES/NDA@SIGMOD/PODS*, 2018.

[35] G. Belyanin, R. Suvorov, and S. Grigorev, "Single-source regular path querying in terms of linear algebra," arXiv:2412.10287v2, 2025. [Online]. Available: https://arxiv.org/abs/2412.10287v2

[36] Y. Chen, M. T. Özsu, G. Xiao, Z. Tang, and K. Li, "GSmart: An efficient SPARQL query engine using sparse matrix algebra," *arXiv preprint arXiv:2106.14038*, 2021.

[37] S. Park, S. Kim, and M.-S. Kim, "cuRPQ: A High-Performance GPU-Based Framework for Processing Regular and Conjunctive Regular Path Queries," *Proc. ACM Manag. Data*, vol. 4, pp. 1–28, 2026.

[38] Q. Liang, D. Ouyang, F. Zhang, J. Yang, X. Lin, and Z. Tian, "Efficient regular simple path queries under transitive restricted expressions," *Proc. VLDB Endow.*, vol. 17, no. 7, pp. 1710–1722, 2024.

[39] H. Wang, X. Wang, M. Ma, and Y. You, "RPQBench: A Benchmark for Regular Path Queries on Graph Data," in *WISE 2024*, 2025, pp. 351–367.

[40] M. Ma, H. Wang, X. Wang, Y. You, and J. Ge, "LD-RPQB: a benchmark for regular path queries based on length distribution," *World Wide Web (WWW)*, vol. 28, no. 5, p. 52, 2025.

[41] W. Martens and T. Trautner, "Dichotomies for evaluating simple regular path queries," *TODS*, vol. 44, no. 4, pp. 16:1–16:46, 2019.

[42] W. Gelade, "Succinctness of regular expressions with interleaving, intersection and counting," *Theor. Comput. Sci.*, vol. 411, no. 31-33, pp. 2987–2998, 2010.

[43] S. Warshall, "A theorem on boolean matrices," *J. ACM*, vol. 9, no. 1, pp. 11–12, 1962.

[44] M. Garey and D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.

[45] A. Caprara, F. Furini, C. Gentile, L. Liberti, and A. Lodi, "Hardness of some optimization problems over correlation polyhedra," *arXiv preprint arXiv:2605.02896*, 2026.

[46] Y. R. Wang, M. Willsey, and D. Suciu, "From binary join to free join," *SIGMOD Rec.*, vol. 53, no. 1, pp. 25–31, 2024.

[47] N. Perera, A. K. Sarker, M. Staylor, G. von Laszewski, K. Shan, S. Kamburugamuve, C. Widanage, V. Abeykoon, T. A. Kanewela, and G. Fox, "In-depth analysis on parallel processing patterns for high-performance dataframes," *Future Gener. Comput. Syst.*, vol. 149, pp. 250–264, 2023.

[48] F. Bourse, M. Lelarge, and M. Vojnovic, "Balanced graph edge partition," in *KDD*, 2014, pp. 1456–1465.

[49] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, "Pregel: a system for large-scale graph processing," in *SIGMOD*, 2010, pp. 135–146.

[50] J. Dinan, D. B. Larkins, P. Sadayappan, S. Krishnamoorthy, and J. Nieplocha, "Scalable work stealing," in *SC*, 2009.

[51] Y. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Comput. Surv.*, vol. 31, no. 4, pp. 406–471, 1999.

[52] A. Buluç, J. Gilbert, and V. B. Shah, "Implementing sparse matrices for graph algorithms," *Graph Algorithms in the Language of Linear Algebra*, pp. 287–313, 2011.

[53] X. Yang, Y. Lei, J. Yang, Y. Zhan, Y. Zheng, L. Mao, Z. Wu, Y. Huang, and Y. Qiu, "Adabool: Sparse boolean matrix processing with adaptive data representation," arXiv:2610.21264, 2026, accepted to ACM SIGMOD 2027. [Online]. Available: https://arxiv.org/abs/2610.21264

[54] W. Fan, W. Yu, J. Xu, J. Zhou, X. Luo, Q. Yin, P. Lu, Y. Cao, and R. Xu, "Parallelizing sequential graph computations," *TODS*, vol. 43, no. 4, pp. 18:1–18:39, 2018.

[55] "Full version," 2026, https://anonymous.4open.science/r/RPQC-C2F4/.

[56] G. Wang, W. Xie, A. J. Demers, and J. Gehrke, "Asynchronous large-scale graph processing made easy," in *CIDR*, 2013.

[57] W. Fan, P. Lu, W. Yu, J. Xu, Q. Yin, X. Luo, J. Zhou, and R. Jin, "Adaptive asynchronous parallelization of graph algorithms," *TODS*, vol. 45, no. 2, pp. 6:1–6:45, 2020.

[58] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, "DBpedia: A nucleus for a web of open data," in *ISWC/ASWC*, 2007, pp. 722–735.

[59] J. Biega, E. Kuzey, and F. M. Suchanek, "Inside YAGO2s: A transparent information extraction architecture," in *WWW*, 2013, pp. 325–328.

[60] O. Erling, A. Averbuch, J. Larriba-Pey, H. Chafi, A. Gubichev, A. Prat, M. Pham, and P. Boncz, "The LDBC social network benchmark: Interactive workload," in *SIGMOD*, 2015, pp. 619–630.

[61] P. Kilpeläinen and R. Tuhkanen, "Regular expressions with numerical occurrence indicators—preliminary results," in *SPLST*, 2003, pp. 163–173.

[62] M. Saleem, M. I. Ali, A. Hogan, Q. Mehmood, and A.-C. Ngonga Ngomo, "LSQ: The linked SPARQL queries dataset," in *ISWC*, vol. 9367, 2015, pp. 261–269.

[63] N. Yakovets, P. Godfrey, and J. Gryz, "Query planning for evaluating SPARQL property paths," in *SIGMOD*, 2016, pp. 1875–1889.

[64] P. Selmer, A. Poulovassilis, and P. Wood, "Implementing flexible operators for regular path queries," in *EDBT/ICDT Workshops*, 2015, pp. 149–156.

[65] A. Gubichev, S. J. Bedathur, and S. Seufert, "Sparqling Kleene: Fast Property Paths in RDF-3X," in *GRADES*, 2013.

[66] Neo4j, Inc., "Cypher manual: Variable-length relationships," Online documentation, 2026. [Online]. Available: https://neo4j.com/docs/cypher-manual/current/patterns/variable-length-patterns/

[67] Apache Jena Project, "ARQ – property paths," Project Documentation, 2026. [Online]. Available: https://jena.apache.org/documentation/query/property_paths.html

[68] OpenLink Software, "Using SPARQL 1.1 property paths in virtuoso," Product Documentation, 2017. [Online]. Available: https://docs.openlinksw.com/virtuoso/rdfsparqlrulepath/

[69] R. Harbi, I. Abdelaziz, P. Kalnis, N. Mamoulis, Y. Ebrahim, and M. Sahli, "Accelerating SPARQL queries by exploiting hash-based locality and adaptive partitioning," *The VLDB Journal*, vol. 25, no. 3, pp. 355–380, 2016.

[70] S. Gurajada, S. Seufert, I. Miliaraki, and M. Theobald, "TriAD: A distributed shared-nothing RDF engine based on asynchronous message passing," in *SIGMOD*, 2014, pp. 289–300.

[71] M. Cossu, M. Färber, and G. Lausen, "PRoST: Distributed execution of SPARQL queries using mixed partitioning strategies," in *EDBT*, 2018, pp. 469–472.

[72] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM J. Sci. Comput.*, vol. 20, no. 1, pp. 359–392, 1998.

APPENDIX

A. Proof of Theorem 1

Upper bound. We design an NP algorithm for DMRP:

(1) guess partition $\mathcal{P} = \{P_1, \ldots, P_l\}$ of source vertices in $R(G)$; and

(2) check whether the revenue is at least K; if so, return true.

It is easy to verify the correctness of the algorithm. Moreover, the algorithm is clearly in NP since verification in step (2) can be done in PTIME; hence DMCP is in NP.

Lower bound. We show the NP-hardness by a reduction from the Linear Exact Cover by 3-Sets (LX3C) problem [44], [45], which is to decide, given a set $U = \{1, \ldots, 3n\}$ and a collection $\mathcal{S}$ of subsets $S_1, \ldots, S_k$ of U such that $|S_i| = 3$ ($i \in [1, k]$) and any two numbers $n_i, n_j \in U$ appear in at most one subset in $\mathcal{S}$, whether there exists an exact cover of U from $\mathcal{S}$, *i.e.,* a collection $\mathcal{S}' \subseteq \mathcal{S}$ such that $\mathcal{S}' = \{S'_1, \ldots, S'_n\}$, $U = S'_1 \cup \ldots \cup S'_n$, $S'_i \cap S'_j \neq \emptyset$ for each $i, j \in [1, n]$ with $i \neq j$. Given U and $\mathcal{S}$, we define a set G_b of vertex pairs to encode each subset in $\mathcal{S}$ and $K = 2n$ to ensure that there exists a partition of source vertices in G_b whose total revenue is at least K if and only if there exists an exact cover of U from $\mathcal{S}$.

We define G_b and K as follows.

(1) The set G_b consists of $3n$ source vertices, one for each element in U, and k target vertices, one for each subset in $\mathcal{S}$. For the vertex pairs, if an element i in U appears in a subset S_j, there exists a vertex pair (u_i, v_j) in G_b, where u_i represents element i and v_j encodes the subset S_j; and

(2) The number K is defined as $K = 2n$.

It is easy to verify that the reduction is in PTIME. We show that there exists an exact cover $\mathcal{S}'$ of U from $\mathcal{S}$ if and only if there exists a partition $\mathcal{P} = \{P_1, \ldots, P_l\}$ of source vertices in G_b whose total revenue is at least K.

($\Rightarrow$) Assume that there exists an exact cover $\mathcal{S}'$ of U from $\mathcal{S}$. We construct a partition $\mathcal{P} = \{P_1, \ldots, P_l\}$ of source vertices in $R(G)$ as follows: when $S'_i \in \mathcal{S}'$ and u_1, u_2, u_3 are vertices representing elements in S'_i, we construct a subset P_i consisting of all pairs $(u, v) \in R(G)$ such that $u \in \{u_1, u_2, u_3\}$ and v is the vertex representing S'_i.

Since each element in U appears in exactly on set in $\mathcal{S}'$, $\mathcal{P}$ is a partition of $R(G)$. It remains to show that its total revenue is at least K. Indeed, given a subset P_i, all source vertices link to the same target vertices, *i.e.,* $|C_{P_i}| = 1$. Since $|P_i| = 3$, the total revenue is $\sum_{i=1}^{n}(|P_i| - 1) \cdot |C_{P_i}| = (3-1)n = 2n$.

($\Leftarrow$) Assume that there exists a partition $\mathcal{P} = \{P_1, \ldots, U_s\}$ of source vertices in G_b such that it total revenue is at least $2n$. We next show that there exists an exact cover $\mathcal{S}'$ of U from $\mathcal{S}$. Given the partition $\mathcal{P}$, it suffices to show that each partition P_i consists of 3 vertex pairs, whose target vertices are the same. Because $\mathcal{P}$ forms a partition of G_b, there exist at least n subsets in $\mathcal{P}$. If so, we can deduce an exact cover $\mathcal{S}'$ as follows: for each subset $P_i \in \mathcal{P}$, let $(u_1, v_j), (u_2, v_j), (u_3, v_j)$ be the vertex pairs in P_i. We select the subset S'_j from $\mathcal{S}$, which is represented by vertex v_j. Because $\mathcal{P}$ is a partition of G_b and there exist n subsets, $\{S'_1, \ldots, S'_n\}$ forms an exact cover of U.

We next show that each subset P_i consists of 3 vertex pairs, whose target vertices are the same. Observe that (1) if a partition P_i consists of 1 or more than 3 vertex pairs, then its revenue is 0, since each target vertex is connected to only three source vertices; and (2) if a partition P_i contains fewer than 3 vertex pairs, then its revenue is at most 1, since any two numbers $n_i, n_j \in U$ appear in at most one subset in $\mathcal{S}$. Assume that $\mathcal{P}$ has PN_i subsets having i vertex pairs ($i \in [2, 3]$). Therefore, $2\mathsf{PN}_2 + 3\mathsf{PN}_3 \leq 3n$, and $\mathsf{PN}_2 \leq \frac{1}{2}(3n - 3\mathsf{PN}_3)$. When there exists a subset in $\mathcal{P}$ having two pairs (*i.e.,* $\mathsf{PN}_2 > 0$), the total revenue of $\mathcal{P}$ is at least $\mathsf{PN}_2 + 2\mathsf{PN}_3 \leq \frac{3}{2}n + \frac{1}{2}\mathsf{PN}_3 < \frac{3}{2}n + \frac{1}{2}n = 2n$, a contradiction. Therefore, each subset P_i consists of three vertex pairs and there exist n subsets in $\mathcal{P}$. Moreover, the three vertex pairs in a subset are connected to the same target vertex; otherwise, the revenue of the subset is 0, and the total revenue is smaller than $2n$, a contradiction. □

B. Parallel scalability of RPQCDEval

We show that RPQCDEval is Parallel scalability with respective to Evaluation. To this end, we show by induction on the structure of Q that PEval and IncEval takes $O(\frac{|Q||V|^3}{n})$, where n is the number of workers.

(1) When Q is a or ε, only PEval suffices to compute $Q(G)$, since the graph is partitioned via edge-cut, and each vertex has all its adjacent edges in each fragment. Then each fragment has $O(\frac{|V|}{n})$ vertices. Then PEval takes $O(\frac{|Q||V|}{n}) \leq O(\frac{|Q||V|^3}{n})$ time. So, RPQCDEval takes $O(\frac{|Q||V|^3}{n})$ time.

(2) When Q is $Q_1|Q_2$, by the induction assumption, computing $Q_1(G)$ and $Q_2(G)$ take $O(\frac{|Q_1||V|^3}{n})$ and $O(\frac{|Q_2||V|^3}{n})$ time, respectively. Because we adopt a DFS traversal to collect vertex pairs, computing $Q(G)$ is to compute both $Q_1(G)$ and $Q_2(G)$. So, computing $Q(G)$ takes $O(\frac{|Q_1||V|^3}{n}) + O(\frac{|Q_2||V|^3}{n}) = O(\frac{(|Q_1|+|Q_2|)|V|^3}{n}) = O(\frac{|Q||V|^3}{n})$ time. Moreover, taking union of $Q_1(G)$ and $Q_2(G)$ does not introduce new communication, then RPQCDEval takes $O(\frac{|Q||V|^3}{n})$ time.

(3) Assume that Q is in the form of $Q_1 \cdot Q_2$. By the induction assumption, computing $Q_1(G)$ and $Q_2(G)$ are in $O(\frac{|Q_1||V|^3}{n})$ and $O(\frac{|Q_2||V|^3}{n})$ time, respectively. Because we adopt a DFS traversal to collect query results of Q on G, IncEval first computes $Q_1(G)$ and then $Q_2(G)$. Moreover, after computing $Q_1(G)$ each fragment sends at most $O(\frac{|V|^2}{n})$ many messages for further processing. Indeed, (a) we partition the graphs using edge-cut, and each fragment has at most $O(\frac{|V|}{n})$ vertices; (b) $Q_1(G)$ has at most $O(|V|^2)$ many vertex pairs; (c) each fragment f_i consists of vertex paris in $Q_1(G)$ whose source vertices are in f_i, and has at most $O(\frac{|V|^2}{n})$ vertex pairs; (d) each fragment f_i only send vertex pairs (u, v) to fragment f_j with u in f_i and v in f_j; and (e) different workers send message independently. Because we directly record the vertex pairs in $Q_1(G)$, compressing the message via CSR or CSC takes $O(\frac{|V|^2}{n})$ time [52]. Therefore,

the time complexity of computing $Q(G)$ is bounded by $O(\frac{|Q_1||V|^3}{n}) + O(\frac{|Q_2||V|^3}{n}) + O(\frac{|V|^2}{n}) \leq O(\frac{|Q||V|^3}{n})$.

(4) When Q is in the form of $Q_1\{m_1, n_1\}$, by the induction assumption, computing $Q_1(G)$ is in $O(\frac{|Q_1||V|^3}{n})$ time. Given $Q_1(G)$, we compute $Q_1\{m_1, n_1\}(G)$ using the recursive doubling strategy as follows:

(a) the recursive doubling strategy takes at most $O(|Q|)$ rounds, one for each bit in binary encodes of m_1 and n_1. Here, $|Q|$ is the sum of $|Q_1|$ and binary encodes of m_1 and n_1; when n is ∞, we rewrite $Q_1\{m_1, n_1\}(G)$ into $R\{m\}(G) \cdot R^*(G)$ and compute $R^*(G)$ using transitive closure of $R(G)$ [43];

(b) in each round IncEval takes $O(\frac{|V|^3}{n})$ time to compute the matrix product for auxiliary RPQs F^i and V^i, since each fragment f_i contains vertex pairs (u, v) with u in f_i, and only computes the submatrix of the matrix product $R(G) \cdot R'(G)$ corresponding vertex pairs (u, v) in $R(G)$ with u in f_1. Because fragment f_i has $O(\frac{|V|}{n})$ source vertices in $R(G)$, IncEval takes $O(\frac{|V|}{n} \cdot |V|^2) = O(\frac{|V|^3}{n})$ time to compute matrix product;

(c) in each round IncEval takes at most $O(\frac{|V|^3}{n})$ time to send messages to other workers or receive messages from other workers. Observe that (i) before computing $R(G) \cdot R'(G)$, a fragment receives $R'(G)$ from other workers, which takes $O(|V|^2) \leq O(\frac{|V|^3}{n})$ times, since the number of vertices in a graph is much larger than the number of workers, *i.e.,* $|V| \gg n$; and (ii) after computing $R(G) \cdot R'(G)$, a fragment f_i sends messages in $O(\frac{|V|^2}{n})$ time, since it only sends vertex pairs with source vertices in f_i, the number of which is bounded by $O(\frac{|V|^2}{n})$. Moreover, moving computation from skew workers to idle workers takes $O(\frac{|V|^2}{n})$ time, which does not affect overall complexity of Algorithm RPQCDEval; and

(d) at last, we compute $Q\{m, n\}(G)$ using the auxiliary RPQs F^i and V^i, which takes $O(|Q| \cdot \frac{|V|^3}{n})$ time.

Putting these together, PEval and IncEval take $O(\frac{(|Q|)|V|^3}{n}) \leq O(\frac{|Q||V|^3}{n})$ time. Therefore, RPQCDEval is parallel scalable *w.r.t.* the sequential algorithm Evaluation.